

# **POC-03 - Real-Time Data Platform**

## **Event Hubs, Databricks, Fabric and Synapse**

---

Beginner implementation and troubleshooting guide

Manual setup, Python commands, streaming validation, SQL/KQL, issues and fixes

# How to Use This Guide

---

This guide reconstructs the completed POC as a repeatable beginner runbook. Follow the sections in order the first time. On later runs, use the troubleshooting, command sequence, and validation checklist as a quick reference.

| Section                 | What it covers                                                       |
|-------------------------|----------------------------------------------------------------------|
| 1. Foundation           | Architecture, prerequisites, project structure, event schema         |
| 2. Azure setup          | Resource group, Event Hubs, ADLS Gen2, Databricks                    |
| 3. Local ingestion      | Virtual environment, .env, send_events.py, receive_events.py         |
| 4. Databricks streaming | Bronze/Silver, checkpoints, watermark, dedup, Serverless fixes       |
| 5. Gold and SQL         | Gold KPIs, Unity Catalog/SQL views, validation queries               |
| 6. Fabric and Synapse   | Eventstream, Eventhouse/KQL, OneLake, serverless SQL reference path  |
| 7. Operations           | Monitoring, Purview notes, cleanup, final validation, interview Q&A; |

# How to Use This Guide - Continued

---

## **Evidence convention**

Screenshots show the actual POC run where supplied. Where the repository contains a reference branch without a screenshot, the guide labels it as a reference implementation step rather than claiming screenshot evidence.

# POC Success State - What Was Actually Proven

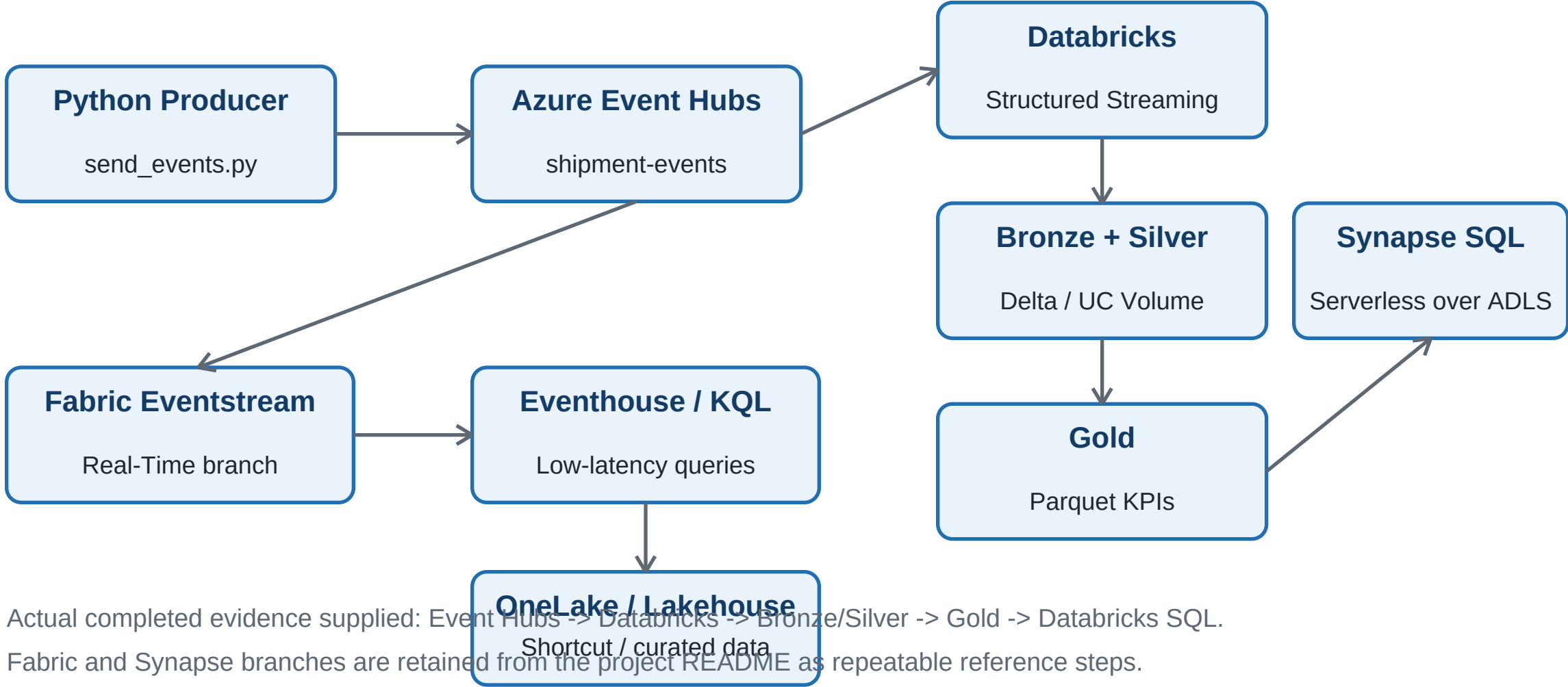
---

- Azure resource group, Event Hubs namespace, storage account, and Azure Databricks workspace were created.
- The Python producer successfully sent shipment events to Azure Event Hubs, and the local receiver successfully read them.
- The Databricks stream ultimately ran successfully on Serverless compute using a Unity Catalog Volume-compatible approach.
- A successful evidence run wrote 50 raw Bronze events.
- Silver validation produced 48 clean events with duplicate count 0.
- Gold KPI aggregation was exported successfully from the 48 clean Silver events.
- Databricks SQL views were created over the Silver Delta path and Gold Parquet path.

## Important architecture difference

The original repository design writes curated data to ADLS with abfss:// paths. The successful Serverless run in the screenshots was adapted to /Volumes/dbw\_logistics\_poc/default/realtime. Synapse serverless SQL can query the repository ADLS path, but it cannot directly use the Databricks managed Volume path. To repeat the Synapse branch, publish Gold to ADLS first.

# Architecture - What You Are Building



# Learning Objectives and Cost Guardrails

---

## You should be able to explain

- What Azure Event Hubs does and why partition keys matter.
- How a Python producer sends JSON events and how a consumer verifies arrival.
- How Databricks Structured Streaming reads the Event Hubs Kafka-compatible endpoint.
- Checkpoint vs watermark, duplicate handling, late-event handling, and restart behavior.
- Bronze, Silver, and Gold layers and why curated Gold data is useful for BI/SQL.
- Eventstream/Eventhouse/KQL vs Lakehouse/Delta use cases.
- OneLake shortcut vs copy, Synapse serverless SQL, semantic measures, monitoring, and Purview governance.

## Cost guardrails

- Send only a few hundred events for learning.
- Use minimal Databricks compute and stop or auto-terminate it when finished.
- Use Fabric Trial/capacity only if available for your account.
- Use Synapse serverless SQL only; do not create a dedicated SQL pool for this POC.
- Keep Purview/Log Analytics optional if account availability or cost is unclear.

# Project Structure

---

## Repository layout

```
POC_03_REALTIME_FABRIC_SYNAPSE/  
|-- .env.example  
|-- README.md  
|-- requirements.txt  
|-- producer/  
|   |-- send_events.py  
|   \-- receive_events.py  
|-- databricks/  
|   |-- stream_to_delta.py  
|   |-- inspect_and_validate.py  
|   \-- export_gold_parquet.py  
|-- fabric/setup_notes.md  
|-- kql/create_shipment_table.kql  
|-- kql/shipment_queries.kql  
|-- synapse/serverless_queries.sql  
|-- semantic_model/measures.dax  
|-- governance/purview_notes.md  
|-- monitoring/monitoring_checklist.md  
|-- docs/azure_portal_setup.md  
|-- docs/troubleshooting.md
```

# Project Structure - Continued

---

```
| -- docs/cleanup.md  
\ -- evidence/README.md
```



# Prerequisites - Before You Create Anything

---

- Azure subscription with permission to create Resource Groups, Storage Accounts, Event Hubs, Databricks, and Synapse workspaces.
- Python 3.9 or newer and pip on the local machine.
- A browser for Azure portal and Databricks workspace access.
- Microsoft Fabric workspace with supported capacity or Trial only if you execute the Fabric branch.
- Recommended local tools: VS Code and Git. Azure CLI is optional for the manual portal-first path.

## **Beginner rule**

Do not start Databricks or Fabric until local Event Hubs send/receive verification works. This reduces the number of systems you must debug at the same time.

# Event Schema

---

## JSON event example

```
{
  "event_id": "evt-001",
  "order_id": 1001,
  "event_type": "SHIPPED",
  "event_ts": "2026-08-28T10:00:00Z",
  "region": "IN-SOUTH",
  "revenue": 1299.50,
  "fulfillment_minutes": 120,
  "producer_seq": 1
}
```

- event\_id is the deduplication key used in Silver.
- event\_ts is the event-time field used for watermark behavior.
- region is also used by the producer as the partition key.
- revenue and fulfillment\_minutes support Gold KPIs and semantic-model measures.

# Recommended Execution Order

---

| Order | Action                                        | Pass condition                        |
|-------|-----------------------------------------------|---------------------------------------|
| 1     | Create resource group                         | Resource group opens in Azure portal  |
| 2     | Create Event Hubs namespace + shipment-events | Event Hub entity exists               |
| 3     | Create ADLS Gen2 storage + realtime container | Container/filesystem exists           |
| 4     | Configure local .env                          | Python can read the connection string |
| 5     | Run producer and receiver                     | Events send and receive successfully  |
| 6     | Create Databricks and import scripts          | Workspace/notebooks open              |
| 7     | Run Bronze/Silver stream                      | Rows written and Delta files created  |
| 8     | Validate Silver                               | No duplicate event_id values          |
| 9     | Build Gold                                    | Gold KPI summary exported             |
| 10    | Create SQL views / queries                    | Curated queries execute               |
| 11    | Fabric/Synapse branch if required             | KQL/OPENROWSET validation succeeds    |

# Step 1 - Create the Azure Resource Group

---

- Open Azure portal and search for Resource groups.
- Select Create.
- Choose your Azure subscription.
- Use a dedicated POC resource group. The completed run used rg-logistics-poc.
- Choose the Azure region you intend to use for the POC resources where practical.
- Select Review + create, then Create.

## Verify

Open the resource group after deployment. It should exist before you create the remaining services.

# Evidence - Resource Group Created

The screenshot displays the Azure Resource Manager (ARM) console interface. On the left, the 'Resource Manager | Resource groups' sidebar is visible, showing a list of resource groups: 'NetworkWatcherRG' and 'rg-logistics-poc'. The 'rg-logistics-poc' group is selected. The main pane shows the 'Overview' tab for the 'rg-logistics-poc' resource group. It includes a search bar, a 'Create' button, and a 'Manage view' dropdown. Below the search bar, there are tabs for 'Essentials', 'Resources', and 'Recommendations'. The 'Resources' tab is active, showing a message: 'No resources match your filters. Try changing or clearing your filters.' There are buttons for '+ Create' and 'Clear filters', and a link for 'Learn more'. At the bottom, it says 'Showing 1 - 0 of 0. Display count: auto'. The footer of the console shows the URL 'portal.azure.com/#home' and a note about pressing 'Ctrl+Shift+F' to add or remove favorites.

Home > Resource Manager | Resource groups

Resource Manager | Resource groups

Default Directory

Search

+ Create Manage view

Resource Manager

All resources

Favorite resources

Recent resources

Resource groups

Tags

Organization

Tools

Deployments

Help

Showing 1 - 2 of 2. Display count: auto

ortal.azure.com/#home pressing Ctrl+Shift+F

rg-logistics-poc

Resource group

Search

+ Create Manage view Delete resource group Refresh Group by none

Export resource groups using Bicep or Terraform +2

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Essentials

Resources Recommendations

Filter for any field...

Type equals all

Location equals all

+ Add filter

No resources match your filters

Try changing or clearing your filters.

+ Create Clear filters

Learn more

Showing 1 - 0 of 0. Display count: auto

Add or remove favorites by pressing Ctrl+Shift+F

Give feedback

The completed run shows the dedicated resource group rg-logistics-poc in Azure Resource Manager.

## Step 2 - Create the Event Hubs Namespace

---

- Search Azure portal for Event Hubs and create a namespace.
- Select the existing POC resource group.
- Use a globally unique namespace name. The completed run used an eh-logistics-poc-dev style name.
- Keep throughput/capacity minimal for the POC.
- The raw setup notes used Standard for the Kafka/multiple-consumer-group learning path.
- Create the namespace and wait for deployment to complete.

### Then create the Event Hub entity

- Open the namespace -> Event Hubs -> + Event Hub.
- Name the entity shipment-events.
- Use a small/default partition count and minimal/default retention for the POC.

# Evidence - Event Hubs Namespace

Home

 **eh-logistics-poc-dev** | Overview

Deployment

Search

×

«

 Delete

 Cancel

 Redeploy

 Download

 Refresh

 Overview

 Inputs

 Outputs

 Template

 Your deployment is complete

 Deployment name : eh-logistics-poc-dev

Subscription : [Azure subscription 1](#)

Resource group : [rg-logistics-poc](#)

Start time : 9/2/2026, 12:07:52 PM

Correlation ID : dd4f2ec0-c77d-403e-8b9e-b862622dc918

> Deployment details

> Next steps

Go to resource

 **Cost management**

Get notified to stay within your budget and prevent unexpected charges on your bill.

[Set up cost alerts >](#)

 **Microsoft Defender for Cloud**

Secure your apps and infrastructure

[Go to Microsoft Defender for Cloud >](#)

**Free Microsoft tutorials**

[Start learning today >](#)

**Work with an expert**

Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.

[Find an Azure expert >](#)

Add or remove favorites by pressing Ctrl+Shift+F

The Event Hubs namespace deployment completed successfully.

# Event Hubs Authentication and Consumer Group

---

## Connection string for the local POC

- Namespace -> Shared access policies.
- Use a POC policy with the rights required by your producer/consumer. The raw run used the namespace shared access policy path.
- Copy Connection string-primary key, not only the Primary key value.
- Store the connection string in .env. Never commit the real secret.

## Event Hub values

### Environment values

```
EVENT_HUB_NAME=shipment-events  
EVENT_HUB_CONSUMER_GROUP=$Default  
EVENT_HUB_NAMESPACE=<your-namespace>  
EVENT_HUB_KAFKA_BOOTSTRAP=<your-namespace>.servicebus.windows.net:9093
```

### Production security

For a production design, prefer Microsoft Entra ID / managed identity where supported rather than long-lived shared keys.



## Step 3 - Create ADLS Gen2 Storage

---

- Azure portal -> Storage accounts -> Create.
- Use the same POC resource group.
- Choose a globally unique lowercase storage account name. The completed run used stlogisticspoc987.
- Performance: Standard.
- For the learning POC, the raw steps selected locally redundant storage (LRS).
- In Advanced settings, enable Hierarchical namespace. This turns on ADLS Gen2 semantics.
- Create the storage account.

### Create the filesystem/container

- Storage account -> Data storage -> Containers -> + Container.
- Name it realtime.
- Keep anonymous access disabled/private.

# Evidence - Storage Account Created

[Home](#)

 **stlogisticspoc987\_1788332728934** | Overview ...

Deployment

×

⏪

Delete

Cancel

Redeploy

Download

Refresh

Overview

Inputs

Outputs

Template

Your deployment is complete

Deployment name : stlogisticspoc987\_1788332728934

Subscription : [Azure subscription 1](#)

Resource group : [rg-logistics-poc](#)

Start time : 9/2/2026, 12:35:37 PM

Correlation ID : a1ed9083-57ac-4b0b-950d-4c719be635f2

> Deployment details

> Next steps

Go to resource

The storage account deployment completed successfully.

# ADLS Paths and .env Storage Values

---

## Repository design - ADLS paths

ADLS\_ACCOUNT\_NAME=stlogisticspoc987

ADLS\_CONTAINER\_NAME=realtime

DELTA\_BRONZE\_PATH=abfss://realtime@stlogisticspoc987.dfs.core.windows.net/  
delta/bronze/shipment\_events

DELTA\_SILVER\_PATH=abfss://realtime@stlogisticspoc987.dfs.core.windows.net/  
delta/silver/shipment\_events

GOLD\_PARQUET\_PATH=abfss://realtime@stlogisticspoc987.dfs.core.windows.net/  
gold/shipment\_summary

## Do not create every subfolder manually

Spark creates Delta/Parquet subdirectories when the write succeeds and permissions are correct.

# Step 4 - Create the Local Python Environment

---

## Windows Command Prompt / PowerShell

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
copy .env.example .env
```

## Linux / macOS / WSL activation

### Linux / macOS / WSL

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

### Required package

The project requirements install azure-eventhub and python-dotenv for the local producer/receiver path.

# Complete .env Template - Safe Placeholder Version

---

## **.env - never place real secrets in shared documentation**

```
EVENT_HUB_CONNECTION_STRING=Endpoint=sb://<namespace>.servicebus.windows.net/;  
    SharedAccessKeyName=<policy>;SharedAccessKey=<key>  
EVENT_HUB_NAME=shipment-events  
EVENT_HUB_CONSUMER_GROUP=$Default  
  
EVENT_COUNT=200  
EVENT_DELAY_SECONDS=0.15  
DUPLICATE_EVERY=25  
LATE_EVENT_EVERY=40  
  
ADLS_ACCOUNT_NAME=<storage-account-name>  
ADLS_CONTAINER_NAME=realtime  
DELTA_BRONZE_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/  
    delta/bronze/shipment_events  
DELTA_SILVER_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/  
    delta/silver/shipment_events  
GOLD_PARQUET_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/  
    gold/shipment_summary  
  
EVENT_HUB_NAMESPACE=<namespace>
```

# Complete .env Template - Safe Placeholder Version - Continued

---

```
EVENT_HUB_KAFKA_BOOTSTRAP=<namespace>.servicebus.windows.net:9093
```

# First Local Verification Commands

---

## 1. Confirm Python can read .env

### PowerShell / CMD

```
python -c "import os; from dotenv import load_dotenv; load_dotenv();  
print('Length of connection string:',  
len(os.getenv('EVENT_HUB_CONNECTION_STRING') or ''))"
```

## 2. Send a small batch first

### Command

```
python producer\send_events.py --count 20 --delay 0.2
```

## 3. Receive a few events

### Command

```
python producer\receive_events.py --max-events 10 --starting-position -1
```

# First Local Verification Commands - Continued

---

## Pass condition

Do not move to Databricks until both sender and receiver work locally.



# Issue 1 - getaddrinfo Failed / Placeholder Namespace

---

## Symptom

Python cannot resolve the host because .env still contains a placeholder such as .servicebus.windows.net.

## Fix

- Open your Event Hubs namespace in Azure portal and copy the exact namespace name.
- Set EVENT\_HUB\_NAMESPACE to that exact value.
- Set EVENT\_HUB\_KAFKA\_BOOTSTRAP to .servicebus.windows.net:9093.
- Replace the placeholder connection string with the real Connection string-primary key.
- Save .env and rerun the producer.

## Issue 2 - Connection String Is Blank or Malformed

```
Traceback (most recent call last):
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 141, in <module>
    main()
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 137, in main
    send_events(args.count, args.delay, args.duplicate_every, args.late_every)
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 70, in send_events
    producer = EventHubProducerClient.from_connection_string(
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_producer_client.py", line 545, in from_connection_string
    constructor_args = cls._from_connection_string(
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_client_base.py", line 329, in _from_connection_string
    host, policy, key, entity, token, token_expiry, emulator = _parse_conn_str(conn_str, **kwargs)
                                                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_client_base.py", line 88, in _parse_conn_str
    conn_settings = core_parse_connection_string(conn_str, case_sensitive_keys=check_case)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\core\utils\_connection_string_parser.py", line 27, in parse_connection_string
    raise ValueError("Connection string is either blank or malformed.")
ValueError: Connection string is either blank or malformed.

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>
```

Terminal evidence from the failed local producer run. Personal local-path details are cropped/anonymized.

## Issue 2 - Connection String Is Blank or Malformed - Continued

---

### Primary cause from the run

Python either could not load `EVENT_HUB_CONNECTION_STRING` from `.env` or the value was not a valid Event Hubs connection string.

# Fix - Validate the Event Hubs Connection String

---

## Check 1 - Copy the right field

- Wrong: only the random-looking Primary key.
- Correct: Connection string-primary key beginning with Endpoint=sb://.

## Check 2 - Verify the Windows filename

### PowerShell

```
Get-ChildItem -Hidden .env*  
# If the file is actually .env.txt:  
Rename-Item .env.txt .env
```

## Check 3 - Keep .env formatting simple

### Correct pattern

```
EVENT_HUB_CONNECTION_STRING=Endpoint=sb://...  
# Avoid extra spaces around the variable name and avoid accidental comments.
```

## Check 4 - Test the loaded length

# Fix - Validate the Event Hubs Connection String - Continued

---

## Diagnostic command

```
python -c "import os; from dotenv import load_dotenv; load_dotenv();  
print(len(os.getenv('EVENT_HUB_CONNECTION_STRING') or ''))"
```

# Success - Producer Sent the Event Batch

```
[168/200] NORMAL      id=evt-9a739fb158ce type=SHIPPED    region=IN-WEST ts=2026-09-02T07:19:04Z
[169/200] NORMAL      id=evt-4389556967d2 type=PACKED    region=IN-EAST ts=2026-09-02T07:19:05Z
[170/200] NORMAL      id=evt-a9576821da96 type=DELAYED    region=IN-WEST ts=2026-09-02T07:19:05Z
[171/200] NORMAL      id=evt-037c0b970c9b type=SHIPPED    region=IN-NORTH ts=2026-09-02T07:19:06Z
[172/200] NORMAL      id=evt-7eb72f74fee9 type=PACKED    region=IN-EAST ts=2026-09-02T07:19:06Z
[173/200] NORMAL      id=evt-a48492969b00 type=CREATED    region=IN-WEST ts=2026-09-02T07:19:06Z
[174/200] NORMAL      id=evt-afc9f3a5cbc7 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:07Z
[175/200] DUPLICATE    id=evt-afc9f3a5cbc7 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:07Z
[176/200] NORMAL      id=evt-b1494312d311 type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:07Z
[177/200] NORMAL      id=evt-0547d6932fb3 type=DELAYED    region=IN-NORTH ts=2026-09-02T07:19:08Z
[178/200] NORMAL      id=evt-83806a05ec90 type=CREATED    region=IN-SOUTH ts=2026-09-02T07:19:08Z
[179/200] NORMAL      id=evt-577d1f2ad115 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:09Z
[180/200] NORMAL      id=evt-a17fa52f752e type=PACKED    region=IN-SOUTH ts=2026-09-02T07:19:09Z
[181/200] NORMAL      id=evt-369c9c532d4f type=IN_TRANSIT region=IN-WEST ts=2026-09-02T07:19:09Z
[182/200] NORMAL      id=evt-08bcf099bbbf type=IN_TRANSIT region=IN-NORTH ts=2026-09-02T07:19:10Z
[183/200] NORMAL      id=evt-6f04c50ba215 type=PACKED    region=IN-SOUTH ts=2026-09-02T07:19:10Z
[184/200] NORMAL      id=evt-1d20a96e32af type=PACKED    region=IN-EAST ts=2026-09-02T07:19:11Z
[185/200] NORMAL      id=evt-95975e9ad9bb type=DELAYED    region=IN-WEST ts=2026-09-02T07:19:11Z
[186/200] NORMAL      id=evt-a3309418b466 type=PACKED    region=IN-NORTH ts=2026-09-02T07:19:11Z
[187/200] NORMAL      id=evt-633a01df7738 type=DELIVERED  region=IN-NORTH ts=2026-09-02T07:19:12Z
[188/200] NORMAL      id=evt-ee02e20ac5b0 type=PACKED    region=IN-EAST ts=2026-09-02T07:19:12Z
[189/200] NORMAL      id=evt-a5161ac6a187 type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:13Z
[190/200] NORMAL      id=evt-184820e1a25f type=DELIVERED  region=IN-WEST ts=2026-09-02T07:19:13Z
[191/200] NORMAL      id=evt-8073c63e6ecd type=SHIPPED    region=IN-NORTH ts=2026-09-02T07:19:13Z
[192/200] NORMAL      id=evt-d267c573fea8 type=SHIPPED    region=IN-WEST ts=2026-09-02T07:19:14Z
[193/200] NORMAL      id=evt-93f37ab95dad type=PACKED    region=IN-SOUTH ts=2026-09-02T07:19:14Z
[194/200] NORMAL      id=evt-000b2a08bd99 type=CREATED    region=IN-WEST ts=2026-09-02T07:19:15Z
[195/200] NORMAL      id=evt-0264ddf8604e type=DELIVERED  region=IN-SOUTH ts=2026-09-02T07:19:15Z
[196/200] NORMAL      id=evt-c264682cdf3f type=SHIPPED    region=IN-EAST ts=2026-09-02T07:19:15Z
[197/200] NORMAL      id=evt-4ed1aa278289 type=DELIVERED  region=IN-WEST ts=2026-09-02T07:19:16Z
[198/200] NORMAL      id=evt-5ecaabd51a4e type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:16Z
[199/200] NORMAL      id=evt-94918d3c3d33 type=PACKED    region=IN-EAST ts=2026-09-02T07:19:17Z
[200/200] DUPLICATE    id=evt-94918d3c3d33 type=PACKED    region=IN-EAST ts=2026-09-02T07:19:17Z

Summary
  sent attempts : 200
  duplicates    : 8
  late events    : 4
Check Event Hubs > Monitoring > Metrics > Incoming Messages.
```

The producer completed successfully and printed NORMAL, DUPLICATE, and LATE event markers.

# Success - Producer Sent the Event Batch - Continued

---

## Observed behavior

The raw run confirmed a successful 200-event producer execution. Later validation batches also used --count 50 and --count 100.

# Verify Event Arrival with receive\_events.py

## Run from the project root

```
python producer\receive_events.py --max-events 20 --starting-position -1
```

- The receiver uses EVENT\_HUB\_CONNECTION\_STRING, EVENT\_HUB\_NAME, and EVENT\_HUB\_CONSUMER\_GROUP from .env.
- starting-position -1 reads from the beginning of retained events for verification.
- Stop with Ctrl+C if you run without a max-events limit.

```
(common-venv-py3.12) C:\Users\ermd\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>python producer/receive_events.py
Receiving up to 20 events from shipment-events (consumer group=$Default, starting_position=-1)
[001] partition=0 offset=0 sequence=0 payload={'event_id': 'evt-8ef4eaaec6ff', 'order_id': 1003, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:17:58Z', 'region': 'IN-WEST', 'revenue': 2977.2, 'fulfillment_minutes': 720, 'producer_seq': 3}
[002] partition=0 offset=296 sequence=1 payload={'event_id': 'evt-485fecfc89bd', 'order_id': 1004, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:17:59Z', 'region': 'IN-WEST', 'revenue': 770.51, 'fulfillment_minutes': 281, 'producer_seq': 4}
[003] partition=0 offset=592 sequence=2 payload={'event_id': 'evt-3fca5b1ab08', 'order_id': 1006, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18:00Z', 'region': 'IN-WEST', 'revenue': 1106.75, 'fulfillment_minutes': 463, 'producer_seq': 6}
[004] partition=1 offset=0 sequence=0 payload={'event_id': 'evt-2ca2fc7fc93b', 'order_id': 1001, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:17:55Z', 'region': 'IN-EAST', 'revenue': 300.13, 'fulfillment_minutes': 304, 'producer_seq': 1}
[005] partition=0 offset=896 sequence=3 payload={'event_id': 'evt-96221f2a0cc2', 'order_id': 1008, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:01Z', 'region': 'IN-WEST', 'revenue': 571.77, 'fulfillment_minutes': 436, 'producer_seq': 8}
[006] partition=1 offset=296 sequence=1 payload={'event_id': 'evt-464de99e4d1e', 'order_id': 1002, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:17:58Z', 'region': 'IN-SOUTH', 'revenue': 2189.46, 'fulfillment_minutes': 349, 'producer_seq': 2}
[007] partition=0 offset=1192 sequence=4 payload={'event_id': 'evt-d10e5fda2a7c', 'order_id': 1010, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:18:01Z', 'region': 'IN-NORTH', 'revenue': 3766.99, 'fulfillment_minutes': 557, 'producer_seq': 10}
[008] partition=1 offset=600 sequence=2 payload={'event_id': 'evt-b304bdc08a43c', 'order_id': 1005, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:17:59Z', 'region': 'IN-SOUTH', 'revenue': 3365.32, 'fulfillment_minutes': 183, 'producer_seq': 5}
[009] partition=0 offset=1496 sequence=5 payload={'event_id': 'evt-3b2a6c307127', 'order_id': 1012, 'event_type': 'IN_TRANSIT', 'event_ts': '2026-09-02T07:18:02Z', 'region': 'IN-WEST', 'revenue': 651.76, 'fulfillment_minutes': 609, 'producer_seq': 12}
[010] partition=1 offset=904 sequence=3 payload={'event_id': 'evt-fcf614256ea0', 'order_id': 1007, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:00Z', 'region': 'IN-SOUTH', 'revenue': 3497.85, 'fulfillment_minutes': 591, 'producer_seq': 7}
[011] partition=0 offset=1800 sequence=6 payload={'event_id': 'evt-a977f79cdeb', 'order_id': 1014, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18:03Z', 'region': 'IN-WEST', 'revenue': 539.8, 'fulfillment_minutes': 707, 'producer_seq': 14}
[012] partition=1 offset=1208 sequence=4 payload={'event_id': 'evt-03c641182a57', 'order_id': 1009, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:01Z', 'region': 'IN-SOUTH', 'revenue': 290.44, 'fulfillment_minutes': 272, 'producer_seq': 9}
[013] partition=0 offset=2104 sequence=7 payload={'event_id': 'evt-011f92bc7c5f', 'order_id': 1015, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:03Z', 'region': 'IN-WEST', 'revenue': 307.1, 'fulfillment_minutes': 405, 'producer_seq': 15}
[014] partition=1 offset=1512 sequence=5 payload={'event_id': 'evt-ff2b71f79507', 'order_id': 1011, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:02Z', 'region': 'IN-SOUTH', 'revenue': 4907.92, 'fulfillment_minutes': 433, 'producer_seq': 11}
[015] partition=0 offset=2400 sequence=8 payload={'event_id': 'evt-a0b30359711f', 'order_id': 1016, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18:04Z', 'region': 'IN-WEST', 'revenue': 625.73, 'fulfillment_minutes': 486, 'producer_seq': 16}
[016] partition=1 offset=1816 sequence=6 payload={'event_id': 'evt-2a3597c0bc28d', 'order_id': 1013, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:02Z', 'region': 'IN-SOUTH', 'revenue': 697.03, 'fulfillment_minutes': 711, 'producer_seq': 13}
[017] partition=0 offset=2704 sequence=9 payload={'event_id': 'evt-66ac2fdad98b', 'order_id': 1018, 'event_type': 'IN_TRANSIT', 'event_ts': '2026-09-02T07:18:05Z', 'region': 'IN-WEST', 'revenue': 4158.48, 'fulfillment_minutes': 573, 'producer_seq': 18}
[018] partition=1 offset=2120 sequence=7 payload={'event_id': 'evt-703eb2169fe2', 'order_id': 1017, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:04Z', 'region': 'IN-SOUTH', 'revenue': 2579.07, 'fulfillment_minutes': 279, 'producer_seq': 17}
[019] partition=0 offset=3008 sequence=10 payload={'event_id': 'evt-be2bf56fe50f', 'order_id': 1021, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:18:05Z', 'region': 'IN-WEST', 'revenue': 1000.0, 'fulfillment_minutes': 0, 'producer_seq': 21}
```

Local consumer output confirms shipment telemetry can be read from Event Hubs.



# Producer Test Flags - Create Duplicates and Late Events

---

## Recommended data-quality test batch

```
python producer\send_events.py --count 200 --delay 0.15 \  
  --duplicate-every 25 --late-every 40
```

- Every 25th event attempt can reuse the previous event\_id so Silver deduplication can be demonstrated.
- Every 40th event can use an event\_ts about 20 minutes old so watermark/late-event behavior can be demonstrated.
- region is used as the Event Hubs partition key in the producer code.

## Step 5 - Create Azure Databricks

---

- Azure portal -> Azure Databricks -> Create.
- Use the existing POC resource group.
- The completed run used a dbw-logistics-poc style workspace name.
- Keep the workspace in the same region as the POC resources where practical.
- Create, then select Launch Workspace.

### Cost

Use the smallest practical compute for your account and stop it when finished. The run shown in later screenshots used Serverless compute.

# Evidence - Databricks Workspace Created

 **rg-logistics-poc\_dbw-logistics-poc** | Overview ...  
Deployment

× <<  Delete  Cancel  Redeploy  Download  Refresh

 Overview

 Inputs

 Outputs

 Template

 **Your deployment is complete**

 Deployment name : rg-logistics-poc\_dbw-logistics-poc  
Subscription : [Azure subscription 1](#)  
Resource group : [rg-logistics-poc](#)

Start time : 9/2/2026, 1:03:54 PM  
Correlation ID : acb87071-6787-4f63-a55c-6be4ca17e082

> Deployment details

✓ Next steps

[Go to resource](#)

Azure Databricks workspace deployment completed successfully.

# Databricks Compute Choice - Understand This Before Running

---

| Option                                   | When it fits this POC                                                   | Important limitation                                                                                                             |
|------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Serverless                               | Convenient and was used by the successful evidence run                  | Does not permit spark.conf.set with fs.azure.account.key and does not support an infinite interactive stream trigger in this run |
| All-purpose single-user cluster          | Useful if you must configure certain Spark filesystem settings directly | Costs run while the cluster is active; use auto-termination                                                                      |
| Unity Catalog external location / volume | Best fit for governed Serverless access                                 | Requires Catalog permissions and correct external/managed storage setup                                                          |

# Import the Databricks Scripts in This Order

---

- Workspace -> your user folder -> Import.
- Import databricks/stream\_to\_delta.py first.
- Import databricks/inspect\_and\_validate.py second.
- Import databricks/export\_gold\_parquet.py third.

## Logical data flow

### Pipeline

Event Hubs Kafka endpoint

- > raw JSON
- > Bronze Delta
- > parse schema
- > event\_ts timestamp
- > watermark + event\_id dedup
- > Silver Delta
- > Gold KPI Parquet

# Issue 3 - Databricks Secret Scope Did Not Exist

The screenshot shows the Databricks notebook interface. The notebook is named 'stream\_to\_delta.py' and is running on a 'Serverless' environment. The code in the notebook is as follows:

```
79  
80 # Write raw events first. The check  
81 bronze_query = (  
82     bronze.writeStream  
83     .format("delta")  
84     .outputMode("append")  
85     .option("checkpointLocation", t  
86     .start(BRONZE_PATH)  
87 )  
88  
89 # COMMAND -----  
90 # 4) Parse + watermark + deduplicate for Silver.  
91 parsed = (  
92     bronze  
93     .withColumn("j", F.from_json("json_body", event_schema))  
94     .select(  
95         F.col("j").getItem(0).as("event"),  
96         F.col("j").getItem(1).as("event_time")  
97     )  
98 )  
99  
100 # Write parsed events to Silver  
101 silver_query = (  
102     parsed.writeStream  
103     .format("delta")  
104     .outputMode("append")  
105     .option("checkpointLocation", t  
106     .start(SILVER_PATH)  
107 )  
108  
109 # COMMAND -----  
110 # 5) Parse + watermark + deduplicate for Gold.  
111 gold_query = (  
112     silver_query.writeStream  
113     .format("delta")  
114     .outputMode("append")  
115     .option("checkpointLocation", t  
116     .start(GOLD_PATH)  
117 )  
118  
119 # COMMAND -----  
120 # 6) Parse + watermark + deduplicate for Platinum.  
121 platinum_query = (  
122     gold_query.writeStream  
123     .format("delta")  
124     .outputMode("append")  
125     .option("checkpointLocation", t  
126     .start(PLATINUM_PATH)  
127 )  
128  
129 # COMMAND -----  
130 # 7) Parse + watermark + deduplicate for Diamond.  
131 diamond_query = (  
132     platinum_query.writeStream  
133     .format("delta")  
134     .outputMode("append")  
135     .option("checkpointLocation", t  
136     .start(DIAMOND_PATH)  
137 )  
138  
139 # COMMAND -----  
140 # 8) Parse + watermark + deduplicate for Emerald.  
141 emerald_query = (  
142     diamond_query.writeStream  
143     .format("delta")  
144     .outputMode("append")  
145     .option("checkpointLocation", t  
146     .start(EMERALD_PATH)  
147 )  
148  
149 # COMMAND -----  
150 # 9) Parse + watermark + deduplicate for Ruby.  
151 ruby_query = (  
152     emerald_query.writeStream  
153     .format("delta")  
154     .outputMode("append")  
155     .option("checkpointLocation", t  
156     .start(RUBY_PATH)  
157 )  
158  
159 # COMMAND -----  
160 # 10) Parse + watermark + deduplicate for Sapphire.  
161 sapphire_query = (  
162     ruby_query.writeStream  
163     .format("delta")  
164     .outputMode("append")  
165     .option("checkpointLocation", t  
166     .start(SAPPHIRE_PATH)  
167 )  
168  
169 # COMMAND -----  
170 # 11) Parse + watermark + deduplicate for Topaz.  
171 topaz_query = (  
172     sapphire_query.writeStream  
173     .format("delta")  
174     .outputMode("append")  
175     .option("checkpointLocation", t  
176     .start(TOPIAZ_PATH)  
177 )  
178  
179 # COMMAND -----  
180 # 12) Parse + watermark + deduplicate for Amethyst.  
181 amethyst_query = (  
182     topaz_query.writeStream  
183     .format("delta")  
184     .outputMode("append")  
185     .option("checkpointLocation", t  
186     .start(AMETHYST_PATH)  
187 )  
188  
189 # COMMAND -----  
190 # 13) Parse + watermark + deduplicate for Garnet.  
191 garnet_query = (  
192     amethyst_query.writeStream  
193     .format("delta")  
194     .outputMode("append")  
195     .option("checkpointLocation", t  
196     .start(GARNET_PATH)  
197 )  
198  
199 # COMMAND -----  
200 # 14) Parse + watermark + deduplicate for Opal.  
201 opal_query = (  
202     garnet_query.writeStream  
203     .format("delta")  
204     .outputMode("append")  
205     .option("checkpointLocation", t  
206     .start(OPAL_PATH)  
207 )  
208  
209 # COMMAND -----  
210 # 15) Parse + watermark + deduplicate for Pearl.  
211 pearl_query = (  
212     opal_query.writeStream  
213     .format("delta")  
214     .outputMode("append")  
215     .option("checkpointLocation", t  
216     .start(PEARL_PATH)  
217 )  
218  
219 # COMMAND -----  
220 # 16) Parse + watermark + deduplicate for Jade.  
221 jade_query = (  
222     pearl_query.writeStream  
223     .format("delta")  
224     .outputMode("append")  
225     .option("checkpointLocation", t  
226     .start(JADE_PATH)  
227 )  
228  
229 # COMMAND -----  
230 # 17) Parse + watermark + deduplicate for Onyx.  
231 onyx_query = (  
232     jade_query.writeStream  
233     .format("delta")  
234     .outputMode("append")  
235     .option("checkpointLocation", t  
236     .start(ONYX_PATH)  
237 )  
238  
239 # COMMAND -----  
240 # 18) Parse + watermark + deduplicate for Quartz.  
241 quartz_query = (  
242     onyx_query.writeStream  
243     .format("delta")  
244     .outputMode("append")  
245     .option("checkpointLocation", t  
246     .start(QUARTZ_PATH)  
247 )  
248  
249 # COMMAND -----  
250 # 19) Parse + watermark + deduplicate for Turquoise.  
251 turquoise_query = (  
252     quartz_query.writeStream  
253     .format("delta")  
254     .outputMode("append")  
255     .option("checkpointLocation", t  
256     .start(TURQUOISE_PATH)  
257 )  
258  
259 # COMMAND -----  
260 # 20) Parse + watermark + deduplicate for Zircon.  
261 zircon_query = (  
262     turquoise_query.writeStream  
263     .format("delta")  
264     .outputMode("append")  
265     .option("checkpointLocation", t  
266     .start(ZIRCON_PATH)  
267 )  
268  
269 # COMMAND -----  
270 # 21) Parse + watermark + deduplicate for Sapphire.  
271 sapphire_query = (  
272     zircon_query.writeStream  
273     .format("delta")  
274     .outputMode("append")  
275     .option("checkpointLocation", t  
276     .start(SAPPHIRE_PATH)  
277 )  
278  
279 # COMMAND -----  
280 # 22) Parse + watermark + deduplicate for Ruby.  
281 ruby_query = (  
282     sapphire_query.writeStream  
283     .format("delta")  
284     .outputMode("append")  
285     .option("checkpointLocation", t  
286     .start(RUBY_PATH)  
287 )  
288  
289 # COMMAND -----  
290 # 23) Parse + watermark + deduplicate for Emerald.  
291 emerald_query = (  
292     ruby_query.writeStream  
293     .format("delta")  
294     .outputMode("append")  
295     .option("checkpointLocation", t  
296     .start(EMERALD_PATH)  
297 )  
298  
299 # COMMAND -----  
300 # 24) Parse + watermark + deduplicate for Topaz.  
301 topaz_query = (  
302     emerald_query.writeStream  
303     .format("delta")  
304     .outputMode("append")  
305     .option("checkpointLocation", t  
306     .start(TOPIAZ_PATH)  
307 )  
308  
309 # COMMAND -----  
310 # 25) Parse + watermark + deduplicate for Amethyst.  
311 amethyst_query = (  
312     topaz_query.writeStream  
313     .format("delta")  
314     .outputMode("append")  
315     .option("checkpointLocation", t  
316     .start(AMETHYST_PATH)  
317 )  
318  
319 # COMMAND -----  
320 # 26) Parse + watermark + deduplicate for Garnet.  
321 garnet_query = (  
322     amethyst_query.writeStream  
323     .format("delta")  
324     .outputMode("append")  
325     .option("checkpointLocation", t  
326     .start(GARNET_PATH)  
327 )  
328  
329 # COMMAND -----  
330 # 27) Parse + watermark + deduplicate for Opal.  
331 opal_query = (  
332     garnet_query.writeStream  
333     .format("delta")  
334     .outputMode("append")  
335     .option("checkpointLocation", t  
336     .start(OPAL_PATH)  
337 )  
338  
339 # COMMAND -----  
340 # 28) Parse + watermark + deduplicate for Pearl.  
341 pearl_query = (  
342     opal_query.writeStream  
343     .format("delta")  
344     .outputMode("append")  
345     .option("checkpointLocation", t  
346     .start(PEARL_PATH)  
347 )  
348  
349 # COMMAND -----  
350 # 29) Parse + watermark + deduplicate for Jade.  
351 jade_query = (  
352     pearl_query.writeStream  
353     .format("delta")  
354     .outputMode("append")  
355     .option("checkpointLocation", t  
356     .start(JADE_PATH)  
357 )  
358  
359 # COMMAND -----  
360 # 30) Parse + watermark + deduplicate for Onyx.  
361 onyx_query = (  
362     jade_query.writeStream  
363     .format("delta")  
364     .outputMode("append")  
365     .option("checkpointLocation", t  
366     .start(ONYX_PATH)  
367 )  
368  
369 # COMMAND -----  
370 # 31) Parse + watermark + deduplicate for Quartz.  
371 quartz_query = (  
372     onyx_query.writeStream  
373     .format("delta")  
374     .outputMode("append")  
375     .option("checkpointLocation", t  
376     .start(QUARTZ_PATH)  
377 )  
378  
379 # COMMAND -----  
380 # 32) Parse + watermark + deduplicate for Turquoise.  
381 turquoise_query = (  
382     quartz_query.writeStream  
383     .format("delta")  
384     .outputMode("append")  
385     .option("checkpointLocation", t  
386     .start(TURQUOISE_PATH)  
387 )  
388  
389 # COMMAND -----  
390 # 33) Parse + watermark + deduplicate for Zircon.  
391 zircon_query = (  
392     turquoise_query.writeStream  
393     .format("delta")  
394     .outputMode("append")  
395     .option("checkpointLocation", t  
396     .start(ZIRCON_PATH)  
397 )  
398  
399 # COMMAND -----  
400 # 34) Parse + watermark + deduplicate for Sapphire.  
401 sapphire_query = (  
402     zircon_query.writeStream  
403     .format("delta")  
404     .outputMode("append")  
405     .option("checkpointLocation", t  
406     .start(SAPPHIRE_PATH)  
407 )  
408  
409 # COMMAND -----  
410 # 35) Parse + watermark + deduplicate for Ruby.  
411 ruby_query = (  
412     sapphire_query.writeStream  
413     .format("delta")  
414     .outputMode("append")  
415     .option("checkpointLocation", t  
416     .start(RUBY_PATH)  
417 )  
418  
419 # COMMAND -----  
420 # 36) Parse + watermark + deduplicate for Emerald.  
421 emerald_query = (  
422     ruby_query.writeStream  
423     .format("delta")  
424     .outputMode("append")  
425     .option("checkpointLocation", t  
426     .start(EMERALD_PATH)  
427 )  
428  
429 # COMMAND -----  
430 # 37) Parse + watermark + deduplicate for Topaz.  
431 topaz_query = (  
432     emerald_query.writeStream  
433     .format("delta")  
434     .outputMode("append")  
435     .option("checkpointLocation", t  
436     .start(TOPIAZ_PATH)  
437 )  
438  
439 # COMMAND -----  
440 # 38) Parse + watermark + deduplicate for Amethyst.  
441 amethyst_query = (  
442     topaz_query.writeStream  
443     .format("delta")  
444     .outputMode("append")  
445     .option("checkpointLocation", t  
446     .start(AMETHYST_PATH)  
447 )  
448  
449 # COMMAND -----  
450 # 39) Parse + watermark + deduplicate for Garnet.  
451 garnet_query = (  
452     amethyst_query.writeStream  
453     .format("delta")  
454     .outputMode("append")  
455     .option("checkpointLocation", t  
456     .start(GARNET_PATH)  
457 )  
458  
459 # COMMAND -----  
460 # 40) Parse + watermark + deduplicate for Opal.  
461 opal_query = (  
462     garnet_query.writeStream  
463     .format("delta")  
464     .outputMode("append")  
465     .option("checkpointLocation", t  
466     .start(OPAL_PATH)  
467 )  
468  
469 # COMMAND -----  
470 # 41) Parse + watermark + deduplicate for Pearl.  
471 pearl_query = (  
472     opal_query.writeStream  
473     .format("delta")  
474     .outputMode("append")  
475     .option("checkpointLocation", t  
476     .start(PEARL_PATH)  
477 )  
478  
479 # COMMAND -----  
480 # 42) Parse + watermark + deduplicate for Jade.  
481 jade_query = (  
482     pearl_query.writeStream  
483     .format("delta")  
484     .outputMode("append")  
485     .option("checkpointLocation", t  
486     .start(JADE_PATH)  
487 )  
488  
489 # COMMAND -----  
490 # 43) Parse + watermark + deduplicate for Onyx.  
491 onyx_query = (  
492     jade_query.writeStream  
493     .format("delta")  
494     .outputMode("append")  
495     .option("checkpointLocation", t  
496     .start(ONYX_PATH)  
497 )  
498  
499 # COMMAND -----  
500 # 44) Parse + watermark + deduplicate for Quartz.  
501 quartz_query = (  
502     onyx_query.writeStream  
503     .format("delta")  
504     .outputMode("append")  
505     .option("checkpointLocation", t  
506     .start(QUARTZ_PATH)  
507 )  
508  
509 # COMMAND -----  
510 # 45) Parse + watermark + deduplicate for Turquoise.  
511 turquoise_query = (  
512     quartz_query.writeStream  
513     .format("delta")  
514     .outputMode("append")  
515     .option("checkpointLocation", t  
516     .start(TURQUOISE_PATH)  
517 )  
518  
519 # COMMAND -----  
520 # 46) Parse + watermark + deduplicate for Zircon.  
521 zircon_query = (  
522     turquoise_query.writeStream  
523     .format("delta")  
524     .outputMode("append")  
525     .option("checkpointLocation", t  
526     .start(ZIRCON_PATH)  
527 )  
528  
529 # COMMAND -----  
530 # 47) Parse + watermark + deduplicate for Sapphire.  
531 sapphire_query = (  
532     zircon_query.writeStream  
533     .format("delta")  
534     .outputMode("append")  
535     .option("checkpointLocation", t  
536     .start(SAPPHIRE_PATH)  
537 )  
538  
539 # COMMAND -----  
540 # 48) Parse + watermark + deduplicate for Ruby.  
541 ruby_query = (  
542     sapphire_query.writeStream  
543     .format("delta")  
544     .outputMode("append")  
545     .option("checkpointLocation", t  
546     .start(RUBY_PATH)  
547 )  
548  
549 # COMMAND -----  
550 # 49) Parse + watermark + deduplicate for Emerald.  
551 emerald_query = (  
552     ruby_query.writeStream  
553     .format("delta")  
554     .outputMode("append")  
555     .option("checkpointLocation", t  
556     .start(EMERALD_PATH)  
557 )  
558  
559 # COMMAND -----  
560 # 50) Parse + watermark + deduplicate for Topaz.  
561 topaz_query = (  
562     emerald_query.writeStream  
563     .format("delta")  
564     .outputMode("append")  
565     .option("checkpointLocation", t  
566     .start(TOPIAZ_PATH)  
567 )  
568  
569 # COMMAND -----  
570 # 51) Parse + watermark + deduplicate for Amethyst.  
571 amethyst_query = (  
572     topaz_query.writeStream  
573     .format("delta")  
574     .outputMode("append")  
575     .option("checkpointLocation", t  
576     .start(AMETHYST_PATH)  
577 )  
578  
579 # COMMAND -----  
580 # 52) Parse + watermark + deduplicate for Garnet.  
581 garnet_query = (  
582     amethyst_query.writeStream  
583     .format("delta")  
584     .outputMode("append")  
585     .option("checkpointLocation", t  
586     .start(GARNET_PATH)  
587 )  
588  
589 # COMMAND -----  
590 # 53) Parse + watermark + deduplicate for Opal.  
591 opal_query = (  
592     garnet_query.writeStream  
593     .format("delta")  
594     .outputMode("append")  
595     .option("checkpointLocation", t  
596     .start(OPAL_PATH)  
597 )  
598  
599 # COMMAND -----  
600 # 54) Parse + watermark + deduplicate for Pearl.  
601 pearl_query = (  
602     opal_query.writeStream  
603     .format("delta")  
604     .outputMode("append")  
605     .option("checkpointLocation", t  
606     .start(PEARL_PATH)  
607 )  
608  
609 # COMMAND -----  
610 # 55) Parse + watermark + deduplicate for Jade.  
611 jade_query = (  
612     pearl_query.writeStream  
613     .format("delta")  
614     .outputMode("append")  
615     .option("checkpointLocation", t  
616     .start(JADE_PATH)  
617 )  
618  
619 # COMMAND -----  
620 # 56) Parse + watermark + deduplicate for Onyx.  
621 onyx_query = (  
622     jade_query.writeStream  
623     .format("delta")  
624     .outputMode("append")  
625     .option("checkpointLocation", t  
626     .start(ONYX_PATH)  
627 )  
628  
629 # COMMAND -----  
630 # 57) Parse + watermark + deduplicate for Quartz.  
631 quartz_query = (  
632     onyx_query.writeStream  
633     .format("delta")  
634     .outputMode("append")  
635     .option("checkpointLocation", t  
636     .start(QUARTZ_PATH)  
637 )  
638  
639 # COMMAND -----  
640 # 58) Parse + watermark + deduplicate for Turquoise.  
641 turquoise_query = (  
642     quartz_query.writeStream  
643     .format("delta")  
644     .outputMode("append")  
645     .option("checkpointLocation", t  
646     .start(TURQUOISE_PATH)  
647 )  
648  
649 # COMMAND -----  
650 # 59) Parse + watermark + deduplicate for Zircon.  
651 zircon_query = (  
652     turquoise_query.writeStream  
653     .format("delta")  
654     .outputMode("append")  
655     .option("checkpointLocation", t  
656     .start(ZIRCON_PATH)  
657 )  
658  
659 # COMMAND -----  
660 # 60) Parse + watermark + deduplicate for Sapphire.  
661 sapphire_query = (  
662     zircon_query.writeStream  
663     .format("delta")  
664     .outputMode("append")  
665     .option("checkpointLocation", t  
666     .start(SAPPHIRE_PATH)  
667 )  
668  
669 # COMMAND -----  
670 # 61) Parse + watermark + deduplicate for Ruby.  
671 ruby_query = (  
672     sapphire_query.writeStream  
673     .format("delta")  
674     .outputMode("append")  
675     .option("checkpointLocation", t  
676     .start(RUBY_PATH)  
677 )  
678  
679 # COMMAND -----  
680 # 62) Parse + watermark + deduplicate for Emerald.  
681 emerald_query = (  
682     ruby_query.writeStream  
683     .format("delta")  
684     .outputMode("append")  
685     .option("checkpointLocation", t  
686     .start(EMERALD_PATH)  
687 )  
688  
689 # COMMAND -----  
690 # 63) Parse + watermark + deduplicate for Topaz.  
691 topaz_query = (  
692     emerald_query.writeStream  
693     .format("delta")  
694     .outputMode("append")  
695     .option("checkpointLocation", t  
696     .start(TOPIAZ_PATH)  
697 )  
698  
699 # COMMAND -----  
700 # 64) Parse + watermark + deduplicate for Amethyst.  
701 amethyst_query = (  
702     topaz_query.writeStream  
703     .format("delta")  
704     .outputMode("append")  
705     .option("checkpointLocation", t  
706     .start(AMETHYST_PATH)  
707 )  
708  
709 # COMMAND -----  
710 # 65) Parse + watermark + deduplicate for Garnet.  
711 garnet_query = (  
712     amethyst_query.writeStream  
713     .format("delta")  
714     .outputMode("append")  
715     .option("checkpointLocation", t  
716     .start(GARNET_PATH)  
717 )  
718  
719 # COMMAND -----  
720 # 66) Parse + watermark + deduplicate for Opal.  
721 opal_query = (  
722     garnet_query.writeStream  
723     .format("delta")  
724     .outputMode("append")  
725     .option("checkpointLocation", t  
726     .start(OPAL_PATH)  
727 )  
728  
729 # COMMAND -----  
730 # 67) Parse + watermark + deduplicate for Pearl.  
731 pearl_query = (  
732     opal_query.writeStream  
733     .format("delta")  
734     .outputMode("append")  
735     .option("checkpointLocation", t  
736     .start(PEARL_PATH)  
737 )  
738  
739 # COMMAND -----  
740 # 68) Parse + watermark + deduplicate for Jade.  
741 jade_query = (  
742     pearl_query.writeStream  
743     .format("delta")  
744     .outputMode("append")  
745     .option("checkpointLocation", t  
746     .start(JADE_PATH)  
747 )  
748  
749 # COMMAND -----  
750 # 69) Parse + watermark + deduplicate for Onyx.  
751 onyx_query = (  
752     jade_query.writeStream  
753     .format("delta")  
754     .outputMode("append")  
755     .option("checkpointLocation", t  
756     .start(ONYX_PATH)  
757 )  
758  
759 # COMMAND -----  
760 # 70) Parse + watermark + deduplicate for Quartz.  
761 quartz_query = (  
762     onyx_query.writeStream  
763     .format("delta")  
764     .outputMode("append")  
765     .option("checkpointLocation", t  
766     .start(QUARTZ_PATH)  
767 )  
768  
769 # COMMAND -----  
770 # 71) Parse + watermark + deduplicate for Turquoise.  
771 turquoise_query = (  
772     quartz_query.writeStream  
773     .format("delta")  
774     .outputMode("append")  
775     .option("checkpointLocation", t  
776     .start(TURQUOISE_PATH)  
777 )  
778  
779 # COMMAND -----  
780 # 72) Parse + watermark + deduplicate for Zircon.  
781 zircon_query = (  
782     turquoise_query.writeStream  
783     .format("delta")  
784     .outputMode("append")  
785     .option("checkpointLocation", t  
786     .start(ZIRCON_PATH)  
787 )  
788  
789 # COMMAND -----  
790 # 73) Parse + watermark + deduplicate for Sapphire.  
791 sapphire_query = (  
792     zircon_query.writeStream  
793     .format("delta")  
794     .outputMode("append")  
795     .option("checkpointLocation", t  
796     .start(SAPPHIRE_PATH)  
797 )  
798  
799 # COMMAND -----  
800 # 74) Parse + watermark + deduplicate for Ruby.  
801 ruby_query = (  
802     sapphire_query.writeStream  
803     .format("delta")  
804     .outputMode("append")  
805     .option("checkpointLocation", t  
806     .start(RUBY_PATH)  
807 )  
808  
809 # COMMAND -----  
810 # 75) Parse + watermark + deduplicate for Emerald.  
811 emerald_query = (  
812     ruby_query.writeStream  
813     .format("delta")  
814     .outputMode("append")  
815     .option("checkpointLocation", t  
816     .start(EMERALD_PATH)  
817 )  
818  
819 # COMMAND -----  
820 # 76) Parse + watermark + deduplicate for Topaz.  
821 topaz_query = (  
822     emerald_query.writeStream  
823     .format("delta")  
824     .outputMode("append")  
825     .option("checkpointLocation", t  
826     .start(TOPIAZ_PATH)  
827 )  
828  
829 # COMMAND -----  
830 # 77) Parse + watermark + deduplicate for Amethyst.  
831 amethyst_query = (  
832     topaz_query.writeStream  
833     .format("delta")  
834     .outputMode("append")  
835     .option("checkpointLocation", t  
836     .start(AMETHYST_PATH)  
837 )  
838  
839 # COMMAND -----  
840 # 78) Parse + watermark + deduplicate for Garnet.  
841 garnet_query = (  
842     amethyst_query.writeStream  
843     .format("delta")  
844     .outputMode("append")  
845     .option("checkpointLocation", t  
846     .start(GARNET_PATH)  
847 )  
848  
849 # COMMAND -----  
850 # 79) Parse + watermark + deduplicate for Opal.  
851 opal_query = (  
852     garnet_query.writeStream  
853     .format("delta")  
854     .outputMode("append")  
855     .option("checkpointLocation", t  
856     .start(OPAL_PATH)  
857 )  
858  
859 # COMMAND -----  
860 # 80) Parse + watermark + deduplicate for Pearl.  
861 pearl_query = (  
862     opal_query.writeStream  
863     .format("delta")  
864     .outputMode("append")  
865     .option("checkpointLocation", t  
866     .start(PEARL_PATH)  
867 )  
868  
869 # COMMAND -----  
870 # 81) Parse + watermark + deduplicate for Jade.  
871 jade_query = (  
872     pearl_query.writeStream  
873     .format("delta")  
874     .outputMode("append")  
875     .option("checkpointLocation", t  
876     .start(JADE_PATH)  
877 )  
878  
879 # COMMAND -----  
880 # 82) Parse + watermark + deduplicate for Onyx.  
881 onyx_query = (  
882     jade_query.writeStream  
883     .format("delta")  
884     .outputMode("append")  
885     .option("checkpointLocation", t  
886     .start(ONYX_PATH)  
887 )  
888  
889 # COMMAND -----  
890 # 83) Parse + watermark + deduplicate for Quartz.  
891 quartz_query = (  
892     onyx_query.writeStream  
893     .format("delta")  
894     .outputMode("append")  
895     .option("checkpointLocation", t  
896     .start(QUARTZ_PATH)  
897 )  
898  
899 # COMMAND -----  
900 # 84) Parse + watermark + deduplicate for Turquoise.  
901 turquoise_query = (  
902     quartz_query.writeStream  
903     .format("delta")  
904     .outputMode("append")  
905     .option("checkpointLocation", t  
906     .start(TURQUOISE_PATH)  
907 )  
908  
909 # COMMAND -----  
910 # 85) Parse + watermark + deduplicate for Zircon.  
911 zircon_query = (  
912     turquoise_query.writeStream  
913     .format("delta")  
914     .outputMode("append")  
915     .option("checkpointLocation", t  
916     .start(ZIRCON_PATH)  
917 )  
918  
919 # COMMAND -----  
920 # 86) Parse + watermark + deduplicate for Sapphire.  
921 sapphire_query = (  
922     zircon_query.writeStream  
923     .format("delta")  
924     .outputMode("append")  
925     .option("checkpointLocation", t  
926     .start(SAPPHIRE_PATH)  
927 )  
928  
929 # COMMAND -----  
930 # 87) Parse + watermark + deduplicate for Ruby.  
931 ruby_query = (  
932     sapphire_query.writeStream  
933     .format("delta")  
934     .outputMode("append")  
935     .option("checkpointLocation", t  
936     .start(RUBY_PATH)  
937 )  
938  
939 # COMMAND -----  
940 # 88) Parse + watermark + deduplicate for Emerald.  
941 emerald_query = (  
942     ruby_query.writeStream  
943     .format("delta")  
944     .outputMode("append")  
945     .option("checkpointLocation", t  
946     .start(EMERALD_PATH)  
947 )  
948  
949 # COMMAND -----  
950 # 89) Parse + watermark + deduplicate for Topaz.  
951 topaz_query = (  
952     emerald_query.writeStream  
953     .format("delta")  
954     .outputMode("append")  
955     .option("checkpointLocation", t  
956     .start(TOPIAZ_PATH)  
957 )  
958  
959 # COMMAND -----  
960 # 90) Parse + watermark + deduplicate for Amethyst.  
961 amethyst_query = (  
962     topaz_query.writeStream  
963     .format("delta")  
964     .outputMode("append")  
965     .option("checkpointLocation", t  
966     .start(AMETHYST_PATH)  
967 )  
968  
969 # COMMAND -----  
970 # 91) Parse + watermark + deduplicate for Garnet.  
971 garnet_query = (  
972     amethyst_query.writeStream  
973     .format("delta")  
974     .outputMode("append")  
975     .option("checkpointLocation", t  
976     .start(GARNET_PATH)  
977 )  
978  
979 # COMMAND -----  
980 # 92) Parse + watermark + deduplicate for Opal.  
981 opal_query = (  
982     garnet_query.writeStream  
983     .format("delta")  
984     .outputMode("append")  
985     .option("checkpointLocation", t  
986     .start(OPAL_PATH)  
987 )  
988  
989 # COMMAND -----  
990 # 93) Parse + watermark + deduplicate for Pearl.  
991 pearl_query = (  
992     opal_query.writeStream  
993     .format("delta")  
994     .outputMode("append")  
995     .option("checkpointLocation", t  
996     .start(PEARL_PATH)  
997 )  
998  
999 # COMMAND -----  
1000 # 94) Parse + watermark + deduplicate for Jade.  
1001 jade_query = (  
1002     pearl_query.writeStream  
1003     .format("delta")  
1004     .outputMode("append")  
1005     .option("checkpointLocation", t  
1006     .start(JADE_PATH)  
1007 )  
1008  
1009 # COMMAND -----  
1010 # 95) Parse + watermark + deduplicate for Onyx.  
1011 onyx_query = (  
1012     jade_query.writeStream  
1013     .format("delta")  
1014     .outputMode("append")  
1015     .option("checkpointLocation", t  
1016     .start(ONYX_PATH)  
1017 )  
1018  
1019 # COMMAND -----  
1020 # 96) Parse + watermark + deduplicate for Quartz.  
1021 quartz_query = (  
1022     onyx_query.writeStream  
1023     .format("delta")  
1024     .outputMode("append")  
1025     .option("checkpointLocation", t  
1026     .start(QUARTZ_PATH)  
1027 )  
1028  
1029 # COMMAND -----  
1030 # 97) Parse + watermark + deduplicate for Turquoise.  
1031 turquoise_query = (  
1032     quartz_query.writeStream  
1033     .format("delta")  
1034     .outputMode("append")  
1035     .option("checkpointLocation", t  
1036     .start(TURQUOISE_PATH)  
1037 )  
1038  
1039 # COMMAND -----  
1040 # 98) Parse + watermark + deduplicate for Zircon.  
1041 zircon_query = (  
1042     turquoise_query.writeStream  
1043     .format("delta")  
1044     .outputMode("append")  
1045     .option("checkpointLocation", t  
1046     .start(ZIRCON_PATH)  
1047 )  
1048  
1049 # COMMAND -----  
1050 # 99) Parse + watermark + deduplicate for Sapphire.  
1051 sapphire_query = (  
1052     zircon_query.writeStream  
1053     .format("delta")  
1054     .outputMode("append")  
1055     .option("checkpointLocation", t  
1056     .start(SAPPHIRE_PATH)  
1057 )  
1058  
1059 # COMMAND -----  
1060 # 100) Parse + watermark + deduplicate for Ruby.  
1061 ruby_query = (  
1062     sapphire_query.writeStream  
1063     .format("delta")  
1064     .outputMode("append")  
1065     .option("checkpointLocation", t  
1066     .start(RUBY_PATH)  
1067 )  
1068  
1069 # COMMAND -----  
1070 # 101) Parse + watermark + deduplicate for Emerald.  
1071 emerald_query = (  
1072     ruby_query.writeStream  
1073     .format("delta")  
1074     .outputMode("append")  
1075     .option("checkpointLocation", t  
1076     .start(EMERALD_PATH)  
1077 )  
1078  
1079 # COMMAND -----  
1080 # 102) Parse + watermark + deduplicate for Topaz.  
1081 topaz_query = (  
1082     emerald_query.writeStream  
1083     .format("delta")  
1084     .outputMode("append")  
1085     .option("checkpointLocation", t  
1086     .start(TOPIAZ_PATH)  
1087 )  
1088  
1089 # COMMAND -----  
1090 # 103) Parse + watermark + deduplicate for Amethyst.  
1091 amethyst_query = (  
1092     topaz_query.writeStream  
1093     .format("delta")  
1094     .outputMode("append")  
1095     .option("checkpointLocation", t  
1096     .start(AMETHYST_PATH)  
1097 )  
1098  
1099 # COMMAND -----  
1100 # 104) Parse + watermark + deduplicate for Garnet.  
1101 garnet_query = (  
1102     amethyst_query.writeStream  
1103     .format("delta")  
1104     .outputMode("append")  
1105     .option("checkpointLocation", t  
1106     .start(GARNET_PATH)  
1107 )  
1108  
1109 # COMMAND -----  
1110 # 105) Parse + watermark + deduplicate for Opal.  
1111 opal_query = (  
1112     garnet_query.writeStream  
1113     .format("delta")  
1114     .outputMode("append")  
1115     .option("checkpointLocation", t  
1116     .start(OPAL_PATH)  
1117 )  
1118  
1119 # COMMAND -----  
1120 # 106) Parse + watermark + deduplicate for Pearl.  
1121 pearl_query = (  
1122     opal_query.writeStream  
1123     .format("delta")  
1124     .outputMode("append")  
1125     .option("checkpointLocation", t  
1126     .start(PEARL_PATH)  
1127 )  
1128  
1129 # COMMAND -----  
1130 # 107) Parse + watermark + deduplicate for Jade.  
1131 jade_query = (  
1132     pearl_query.writeStream  
1133     .format("delta")  
1134     .outputMode("append")  
1135     .option("checkpointLocation", t  
1136     .start(JADE_PATH)  
1137 )  
1138  
1139 # COMMAND -----  
1140 # 108) Parse + watermark + deduplicate for Onyx.  
1141 onyx_query = (  
1142     jade_query.writeStream  
1143     .format("delta")  
1144     .outputMode("append")  
1145     .option("checkpointLocation", t  
1146     .start(ONYX_PATH)  
1147 )  
1148  
1149 # COMMAND -----  
1150 # 109) Parse +
```

## Issue 3 - Databricks Secret Scope Did Not Exist - Continued

---

### Error

Secret does not exist with scope: poc03 and key: event-hub-connection-string.

# Fix - Event Hubs Credential Options in Databricks

---

## Preferred

- Create/use a Databricks secret scope named poc03.
- Store the Event Hubs connection string with key event-hub-connection-string.
- Keep dbutils.secrets.get(...) in the notebook.

## POC-only emergency path used during troubleshooting

### Use placeholders in saved code

```
EVENT_HUB_CONNECTION_STRING = (  
    "Endpoint=sb://eh-logistics-poc-dev.servicebus.windows.net/;"  
    "SharedAccessKeyName=<policy>;SharedAccessKey=<secret>"  
)
```

### Security warning

Do not commit a real connection string. If a secret was exposed in code or a screenshot, rotate it.



# Storage Access Key - Where It Was Found

Home > Storage center | Blob Storage > stlogisticspoc987

Storage center | Blob Storage

[account redacted]

Search

SummaryResources

Overview

All storage resources

Object storage

File storage

Block storage

Data management

Migration

Partner solutions

Management services

Help

CreateRestore...

Name ↑

stlogisticspoc987

Showing 1 - 1 of 1. Display count: auto

Add or remove favorites by pressing Ctrl+Shift+F

stlogisticspoc987 | Access keys

Storage account

Search

Set rotation reminderRefreshGive feedback

Storage browser

Partner solutions

Resource visualizer

Data storage

Containers

Classic file shares

Queues

Tables

Security + networking

Networking

Access keys

Shared access signature

Encryption

Microsoft Defender for Cloud

Data management

Settings

Add or remove favorites by pressing Ctrl+Shift+F

Access keys authenticate your applications' requests to this storage account. Keep your keys in a secure location like Azure Key Vault, and replace them often with new keys. The two keys allow you to replace one while still using the other.

Remember to update the keys with any Azure resources and apps that use this storage account.  
[Learn more about managing storage account access keys](#)

Storage account name  
stlogisticspoc987

key1

Rotate key

Last rotated: 2/9/2026 (0 days ago)

Key  
.....

Show

Connection string  
.....

Show

key2

Rotate key

Last rotated: 2/9/2026 (0 days ago)

Key  
.....

Show

Connection string  
.....

Show

Azure Storage access-key screen used during troubleshooting. Account identity information is redacted.

# Storage Access Key - Where It Was Found - Continued

---

## **Do not copy this design into production**

The storage key path was used only while diagnosing the POC. Serverless later rejected runtime `fs.azure.account.key` configuration, and the working path moved to Unity Catalog Volume storage.

# Issue 4 - JVM Was Not Initialized / CONFIG\_NOT\_AVAILABLE

The screenshot shows a Databricks notebook interface with a Python script. The script configures Spark to connect to an Azure Storage account (ADLS Gen2) using a container named 'realtime'. The configuration includes setting the storage account key and defining paths for Delta and checkpoint data. The output pane displays an error message indicating that the JVM was not initialized and the configuration for the storage account key is not available.

```
15 # Storage Account Details
16 (STORAGE_ACCOUNT_KEY REDACTED)
17 Portal > Storage Account > Access keys > Key 1
18 CONTAINER_NAME = "realtime"
19
20 # Configure Spark to authenticate with ADLS Gen2
21 spark.conf.set(
22     f"fs.azure.account.key.{STORAGE_ACCOUNT_NAME}.dfs.core.windows.net",
23     STORAGE_ACCOUNT_KEY
24 )
25
26 # ADLS Gen2 Delta & Checkpoint Paths
27 BRONZE_PATH = f"abfss://{CONTAINER_NAME}@{STORAGE_ACCOUNT_NAME}.dfs.core.windows.net/delta/bronze/shipment_events"
28 BRONZE_CHECKPOINT = f"abfss://{CONTAINER_NAME}@{STORAGE_ACCOUNT_NAME}.dfs.core.windows.net/checkpoints/bronze/shipment_events"
29
```

Output:

```
Tried to attach usage logger `pyspark.databricks.pandas.usage_logger`, but an exception was raised: JVM wasn't initialised. Did you call it on executor side?
> [CONFIG_NOT_AVAILABLE.WITHOUT_SUGGESTION] Configuration fs.azure.account.key.stlogisticspoc987.dfs.core.windows.net is not available. SQLSTATE: 42K0I
```

The actual storage key shown in the original screenshot has been redacted. The important error is CONFIG\_NOT\_AVAILABLE for fs.azure.account.key on Serverless.

## Issue 4 - JVM Was Not Initialized / CONFIG\_NOT\_AVAILABLE - Conti

---

### Root cause

Serverless compute did not allow `spark.conf.set("fs.azure.account.key..dfs.core.windows.net", ...)`. The JVM warning was secondary to this configuration failure.

# Fix - Choose One Storage Access Pattern

---

## Option A - All-purpose single-user compute

- Attach the notebook to a non-Serverless single-user cluster.
- Use the supported ADLS authentication pattern for that compute.
- Prefer secrets/managed identity over hardcoded keys.

## Option B - Serverless with Unity Catalog (working direction)

- Remove `spark.conf.set(fs.azure.account.key...)`.
- Use a Unity Catalog Volume or a governed external location/credential.
- Grant the appropriate catalog/storage permissions.

### What the successful evidence used

The run moved to `/Volumes/dbw_logistics_poc/default/realtime`, which is why data appeared in Databricks Catalog rather than in the standalone `stlogisticspoc987` container.

# Final Working Storage Paths for the Serverless Run

---

## Working notebook adaptation from the POC run

```
VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"

BRONZE_PATH = f"{VOLUME_BASE}/delta/bronze/shipment_events"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"
GOLD_PATH = f"{VOLUME_BASE}/gold/shipment_summary"

BRONZE_CHECKPOINT = (
    f"{VOLUME_BASE}/checkpoints/v3/bronze/shipment_events"
)
SILVER_CHECKPOINT = (
    f"{VOLUME_BASE}/checkpoints/v3/silver/shipment_events"
)
```

## Why v3 appears here

The checkpoint location was changed during troubleshooting after an earlier zero-row attempt had already committed stream progress. For a normal restart test, keep the same working checkpoint; do not keep changing it.

# Issue 5 - INFINITE\_STREAMING\_TRIGGER\_NOT\_SUPPORTED

---

## Root cause

The Serverless interactive environment in this run did not support an infinite ProcessingTime-style stream. It required a bounded micro-batch trigger.

## Bronze fix

### Bronze - availableNow

```
bronze_query = (  
    bronze.writeStream  
        .format("delta")  
        .outputMode("append")  
        .option("checkpointLocation", BRONZE_CHECKPOINT)  
        .trigger(availableNow=True)  
        .start(BRONZE_PATH)  
)  
bronze_query.awaitTermination()
```

# Issue 5 Fix - Silver availableNow Trigger

---

## Silver - availableNow

```
silver_query = (  
    silver.writeStream  
        .format("delta")  
        .outputMode("append")  
        .option("checkpointLocation", SILVER_CHECKPOINT)  
        .trigger(availableNow=True)  
        .start(SILVER_PATH)  
)  
  
print("Bronze query id:", bronze_query.id)  
print("Silver query id:", silver_query.id)  
silver_query.awaitTermination()  
print("Streaming batch completed successfully!")
```

## Behavior change

availableNow=True is a one-time sweep of data available to the stream. It is not a continuously listening background stream. Send the events before you launch the sweep, or run another sweep afterward.



# Issue 6 - 0 Rows and an Empty Azure Storage Container

---

- The notebook had already been changed to /Volumes/... paths, so writes no longer targeted the standalone ADLS container shown in Azure portal.
- availableNow=True checked the source before the new 50-event batch was sent, so that run processed zero available records.
- A previous zero-row execution had already created checkpoint state, so rerunning with the same temporary test state could skip work you expected to see.

## Key lesson

Always know both (1) where your output path points and (2) whether your trigger is continuous or bounded.

# Evidence - Initial Validation Showed 0 Rows

Microsoft Azure databricks

Search data, notebooks, recents, and more...

CTRL (user redacted)

Finished

compute

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Home

stream\_to\_delta.py

inspect\_and\_validate.py

+

Just now (6s) Python Last edit was now

```
1 # Databricks notebook source
2 from pyspark.sql import functions as F
3
4 SILVER_PATH = "/Volumes/dbw_logistics_poc/default/realtime/delta/silver/shipment_events"
5
6 df = spark.read.format("delta").load(SILVER_PATH)
7
8 print("Silver rows:", df.count())
9 print("Distinct event_id:", df.select("event_id").distinct().count())
10
11 display(
12     df.groupBy("event_type")
13         .count()
14         .orderBy(F.desc("count"))
15 )
16
```

Add parameter

Output Terminal Debugger pip logs

Silver rows: 0  
Distinct event\_id: 0

Table +

| event_type | count |
|------------|-------|
|------------|-------|

Hide performance (6)

| Statement                                                 | Started At             | Tasks         | Dur |
|-----------------------------------------------------------|------------------------|---------------|-----|
| distinct_count = df.select("event_id").distinct().count() | Sep 02, 2026, 05:08 PM | 1/1 completed | 120 |

ID: e4b2407e-ba28-4dc2-aea4-4a05aae9de73

distinct\_count = df.select("event\_id").distinct().count()  
See query text

Query wall-clock duration ⓘ

Total wall-clock duration 200 ms

Start time Sep 02, 2026, 05:08:07 PM GMT+05:30

End time Sep 02, 2026, 05:08:07 PM GMT+05:30

Result fetching by client ⓘ 94 ms

Bytes read 0

Bytes written 0

Rows read 0

Rows written 0

Files read 0

Files written 0

See query profile >

See longest operations for this query

Query Source

inspect\_and\_validate.py / Cell(ID: 465...7170774): 29

See query profile

An intermediate validation attempt showed zero Silver rows. This screenshot is retained because it demonstrates the failure state before the final fix.

# Fix - Correct Order for the Serverless availableNow Test

---

- Interrupt any notebook cell that is still waiting unexpectedly.
- For the troubleshooting rerun, move to fresh checkpoint paths (v3) only because the earlier test state was disposable.
- Keep Kafka startingOffsets set to earliest for the fresh test path if you want retained messages to be considered.
- Send the test events first.
- Then clear state/outputs if needed and Run all so availableNow processes data that is already waiting.

## Troubleshooting sequence

```
python producer\send_events.py --count 50
```

```
# Then run stream_to_delta.py with:
```

```
"startingOffsets": "earliest"
```

```
BRONZE_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/bronze/shipment_events"
```

```
SILVER_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/silver/shipment_events"
```

# Success - Bronze Stream Wrote 50 Rows

Microsoft Azure

Search data, notebooks, recents, and more...

CTRL  
(user redacted)

Finished

compute

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Home

stream\_to\_delta.py

Interrupt 09:29 Python Last edit was 11 minutes ago

```
1 # Databricks notebook source
2 # POC-03: Azure Event Hubs -> Databricks Structured Streaming -> Delta Lake (Serverless Compatible)
3
4 from pyspark.sql import functions as F
5 from pyspark.sql.types import (
6     StructType, StructField, StringType, LongType, DoubleType
7 )
8
9 # COMMAND -----
10 # 1) Configuration & Unity Catalog Volume Setup
11
12 # Ensure the Unity Catalog managed volume exists
13 spark.sql("CREATE VOLUME IF NOT EXISTS dbw_logistics_poc.default.realtime")
14
15 # Event Hub settings
16 EVENT HUB NAMESPACE = "eh-logistics-poc-dev"
```

Output Terminal Debugger pip logs

Hide performance Statements 2/2 Tasks 2/2

| Statement                                       | Started At             | Tasks         | Dur |
|-------------------------------------------------|------------------------|---------------|-----|
| L87 > bronze_query = ( bronze.writeStream .f... | Sep 02, 2026, 04:45 PM | 2/2 completed |     |

ID: 5bd925cc-7bee-4d01-880e-60fb6857133d

bronze\_query = ( bronze.writeStream .format("delta") .outp  
See query text

Query wall-clock duration ⓘ  
Total wall-clock duration 18 s 724 ms  
>

Start time Sep 02, 2026, 04:45:47 PM GMT+05:30  
End time Sep 02, 2026, 04:46:06 PM GMT+05:30  
Result fetching by client ⓘ 0 ms

Bytes read 0

Bytes written 9.45 KB

Rows read 0

Rows written 50

Files read 0

Files written 0

See query profile >

See longest operations for this query

Query Source  
> stream\_to\_delta.py / Cell(ID: 868...9226319): 87

See query profile

Databricks Serverless query profile shows the Bronze write finishing successfully with 50 rows written. User identity is redacted.

# Verify the Files in Databricks Catalog

The screenshot displays the Databricks Catalog Explorer interface. On the left, a sidebar contains navigation options: New, Workspace, Recents, Catalog (selected), Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, and Visual Data Prep. The main panel is divided into two sections. The top section, titled 'Catalog', shows a tree view of the catalog structure: My organization > dbw\_logistics\_poc > default > Volumes (1) > realtime (selected). The bottom section, titled 'realtime', shows the volume's details. It includes a search bar, a description, and a table of files and directories. The table has columns for Name, Size, and Last modified. The files listed are: 000000000000000000000000.crc (2.76 KB, 1 hour ago), 000000000000000000000000.json (1.88 KB, 1 hour ago), 0000000000000000000000001.crc (2.84 KB, 1 hour ago), and 0000000000000000000000001.json (843.00 B, 1 hour ago). There are also two directories: \_\_tmp\_path\_dir and \_staged\_commits. The interface also shows a 'Share' button and an 'Upload to this volume' button.

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

dbw-logistics-poc

+ New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Catalog

No available computes found.

Catalog

Type to search

My organization

dbw\_logistics\_poc

default

Volumes (1)

realtime

information\_schema

system

Shares received

samples

realtime

Overview Files Details Permissions

Description

AI generate Add

/Volumes/dbw\_logistics\_poc/default/realtime / delta / silver / shipment\_events / \_delta\_log

Refresh Create directory

Filter files and directories at this level

| Name                           | Size     | Last modified |
|--------------------------------|----------|---------------|
| 000000000000000000000000.crc   | 2.76 KB  | 1 hour ago    |
| 000000000000000000000000.json  | 1.88 KB  | 1 hour ago    |
| 0000000000000000000000001.crc  | 2.84 KB  | 1 hour ago    |
| 0000000000000000000000001.json | 843.00 B | 1 hour ago    |
| __tmp_path_dir                 |          |               |
| _staged_commits                |          |               |

About this volume

Owner irshad alam

Catalog Explorer shows the realtime Volume and Delta output folders, including Delta transaction-log and Parquet data files.

# Verify the Files in Databricks Catalog - Continued

---

## Why Azure portal looked empty

The successful path shown here is a Databricks Unity Catalog Volume, not the standalone ADLS container.

# Bronze vs Silver - What Each Layer Means

---

| Layer  | Purpose             | POC behavior                                                          |
|--------|---------------------|-----------------------------------------------------------------------|
| Bronze | Raw event capture   | Keeps raw JSON plus Event Hubs/Kafka metadata and ingestion timestamp |
| Silver | Clean event table   | Parses schema, converts event_ts, applies watermark/dedup logic       |
| Gold   | Business aggregates | Produces regional order/revenue/delay/fulfillment KPIs for SQL/BI     |

## Checkpoint vs watermark

- Checkpoint = recovery/progress/state persistence for the streaming query.
- Watermark = event-time threshold used to bound state for late data.
- They solve different problems and should not be treated as interchangeable.

# Run inspect\_and\_validate.py

---

## Core Silver checks

```
SILVER_PATH = (  
    "/Volumes/dbw_logistics_poc/default/realtime/"  
    "delta/silver/shipment_events"  
)  
  
df = spark.read.format("delta").load(SILVER_PATH)  
print("Silver rows:", df.count())  
print("Distinct event_id:", df.select("event_id").distinct().count())  
  
duplicates = (  
    df.groupBy("event_id").count().filter("count > 1").count()  
)  
print("Duplicate count (must be 0):", duplicates)
```



# Success - Silver Validation Metrics

The screenshot shows the Databricks interface with a Python notebook titled 'inspect\_and\_validate.py'. The notebook code includes the following steps:

```
45 df_parsed.write.format("delta").mode("overwrite").save(SILVER_PATH)
46 print("2. Silver Delta table successfully populated!")
47
48 # 6. Validate Silver metrics
49 df_silver = spark.read.format("delta").load(SILVER_PATH)
50 print(f"3. Total clean events in Silver: {df_silver.count()}")
51
52 duplicates = df_silver.groupBy("event_id").count().filter("count > 1").count()
53 print(f"4. Duplicate count (must be 0): {duplicates}")
54
55 # 7. Display breakdown by event_type
56 display(
57     df_silver.groupBy("event_type")
58         .count()
59         .orderBy(F.desc("count"))
60 )
```

The output of the notebook shows the following results:

```
> 3 dataframes: df_bronze, df_parsed, df_silver
1. Total raw events in Bronze: 50
2. Silver Delta table successfully populated!
3. Total clean events in Silver: 48
4. Duplicate count (must be 0): 0
```

The interface also shows a sidebar with navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The top bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and user information.

Final validation evidence: 50 raw Bronze events, Silver populated with 48 clean events, duplicate count 0.

# Success - Silver Validation Metrics - Continued

---

## Why 50 became 48

The producer deliberately injected duplicate event\_id values. Silver is the curated layer, so the final clean count can be lower than the raw Bronze count.

# Build the Gold KPI Layer

---

## Gold aggregation - working Volume path

```
VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"
GOLD_PATH = f"{VOLUME_BASE}/gold/shipment_summary"

df_silver = spark.read.format("delta").load(SILVER_PATH)

gold_kpi_summary = (
    df_silver.groupBy("region")
    .agg(
        F.count("order_id").alias("total_events"),
        F.countDistinct("order_id").alias("unique_orders"),
        F.round(F.sum("revenue"), 2).alias("total_revenue"),
        F.round(F.avg("fulfillment_minutes"), 1).alias("avg_fulfillment_mins"),
        F.count(F.when(F.col("event_type") == "DELAYED", 1)).alias("delayed_shipments"),
        F.count(F.when(F.col("event_type") == "DELIVERED", 1)).alias("delivered_shipments"),
    )
)

gold_kpi_summary.write.mode("overwrite").parquet(GOLD_PATH)
```

# Success - Gold KPI Export

Microsoft Azure

databricks

Search data, notebooks, recents, and more...

CTRL + P

dbw-logistics-poc

New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

Home

stream\_to\_delta.py

inspect\_and\_validate.py

export\_gold\_parquet.py

1 minute ago (14s)

Python

Last edit was 2 minutes ago

Generate

Serverless

Schedule

Comments

Share

10

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

.agg()

F.count("order\_id").alias("total\_events"),

F.countDistinct("order\_id").alias("unique\_orders"),

F.round(F.sum("revenue"), 2).alias("total\_revenue"),

F.round(F.avg("fulfillment\_minutes"), 1).alias("avg\_fulfillment\_mins"),

F.count(F.when(F.col("event\_type") == "DELAYED", 1)).alias("delayed\_shipments"),

F.count(F.when(F.col("event\_type") == "DELIVERED", 1)).alias("delivered\_shipments"),

)

.withColumn(

"on\_time\_delivery\_pct",

F.round(

(F.col("delivered\_shipments") / (F.col("delivered\_shipments") + F.col("delayed\_shipments"))) \* 100,

2,

),

)

.orderBy(F.desc("total\_revenue"))

Add parameter

Output

Terminal

Debugger

pip logs

> 2 dataframes: df\_silver, gold\_kpi\_summary

Loaded 48 clean events from Silver.

Gold KPI summary exported successfully!

Table

+

region

total\_events

unique\_orders

total\_revenue

avg\_fulfillment\_mins

delayed\_shipments

delivered\_shipments

on\_tir

> See performance (3)

The Gold notebook loaded 48 clean Silver events and exported the KPI summary successfully.

# Create SQL Views over Silver and Gold

---

## Databricks SQL

```
CREATE OR REPLACE VIEW dbw_logistics_poc.default.silver_shipment_events AS  
SELECT * FROM delta.`/Volumes/dbw_logistics_poc/default/realtime/  
delta/silver/shipment_events`;
```

```
CREATE OR REPLACE VIEW dbw_logistics_poc.default.gold_shipment_summary AS  
SELECT * FROM parquet.`/Volumes/dbw_logistics_poc/default/realtime/  
gold/shipment_summary`;
```

## Why views were used

The supplied SQL evidence creates views directly over the Volume file locations so the curated data can be queried from Databricks SQL.

# Evidence - SQL Views Created

The screenshot displays the Databricks SQL Editor interface. The left sidebar contains navigation options: Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL (selected), SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, and SQL Warehouses. The main editor area shows a SQL query with two parts: creating a view over the Silver Delta table and creating a view over the Gold Parquet table. The query is executed successfully, as indicated by the 'Run all (1000)' button and the 'verify\_data' status. The results pane at the bottom shows 'Results 2 of 2' and a table view.

```
1 -- 1. Create a View over the Silver Delta table
2 CREATE OR REPLACE VIEW dbw_logistics_poc.default.silver_shipment_events AS
3 SELECT * FROM delta.`/Volumes/dbw_logistics_poc/default/realtime/delta/silver/shipment_events`;
4
5 -- 2. Create a View over the Gold Parquet table
6 CREATE OR REPLACE VIEW dbw_logistics_poc.default.gold_shipment_summary AS
7 SELECT * FROM parquet.`/Volumes/dbw_logistics_poc/default/realtime/gold/shipment_summary`;
```

Results 2 of 2

| Table |
|-------|
| OK    |

Databricks SQL Editor shows the Silver Delta and Gold Parquet views being created successfully.

# Evidence - Curated Data Registration

The screenshot displays the Databricks SQL Editor interface. The top navigation bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and the user profile 'dbw-logistics-poc'. The left sidebar shows a navigation menu with options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. The main editor area shows a SQL query with two steps: creating a view over a Silver Delta table and creating a view over a Gold Parquet table. The query is as follows:

```
1 -- 1. Create a View over the Silver Delta table
2 CREATE OR REPLACE VIEW dbw_logistics_poc.default.silver_shipment_events AS
3 SELECT * FROM delta.`/Volumes/dbw_logistics_poc/default/realtime/delta/silver/shipment_events`;
4
5 -- 2. Create a View over the Gold Parquet table
6 CREATE OR REPLACE VIEW dbw_logistics_poc.default.gold_shipment_summary AS
7 SELECT * FROM parquet.`/Volumes/dbw_logistics_poc/default/realtime/gold/shipment_summary`;
```

Below the query editor, there is a 'Results' section showing 'Results 2 of 2' and a 'Table' tab. The results area is currently empty, displaying 'OK'.

Second curated-data evidence screenshot retained from the supplied POC screenshot set.

# Query the Gold KPI Summary

---

## Databricks SQL

```
SELECT
    region,
    unique_orders,
    total_revenue,
    avg_fulfillment_mins,
    delayed_shipments,
    delivered_shipments,
    on_time_delivery_pct
FROM dbw_logistics_poc.default.gold_shipment_summary
ORDER BY total_revenue DESC;
```

Use this query to validate the final regional KPI dataset after Gold export. If your Gold implementation does not contain `on_time_delivery_pct`, add that computed column in the Gold notebook before running this exact projection.



# Query Detailed Silver Telemetry

---

## Databricks SQL

```
SELECT
    event_type,
    region,
    count(*) AS event_count,
    round(avg(revenue), 2) AS avg_revenue,
    min(event_ts) AS earliest_event,
    max(event_ts) AS latest_event
FROM dbw_logistics_poc.default.silver_shipment_events
GROUP BY event_type, region
ORDER BY event_count DESC;
```

## Pass condition

The query returns grouped counts/timestamps from the deduplicated Silver layer without duplicate event\_id values.

# Restart and Checkpoint Test

---

- Record the current Silver row count.
- Stop the streaming notebook/query.
- Send a new small batch, for example 20 events.
- Restart using the SAME working checkpoint paths.
- Wait for the new batch to complete.
- Run Silver validation again.

## New batch

```
python producer\send_events.py --count 20 --delay 0.15
```

## Critical distinction

For the intentional recovery test, keep the checkpoint. Changing/deleting the checkpoint creates a different stream history and is not a valid demonstration of normal restart recovery.

# Fabric Branch - Workspace, Eventhouse and Eventstream

---

## Evidence status

These steps are present in the repository README/setup notes. The supplied screenshot set does not contain Fabric evidence, so this section is a repeatable reference branch rather than screenshot-validated proof.

- Create a Fabric-enabled workspace or Trial workspace if your tenant allows it.
- Create an Eventhouse and use its KQL database.
- Create the ShipmentEvents KQL table.
- Create an Eventstream and add Azure Event Hubs as the source.
- Select the shipment-events Event Hub and appropriate connection/consumer group.
- Add the Eventhouse/KQL table as the destination.
- Map the JSON fields, then Publish the Eventstream.
- Verify live data and run `ShipmentEvents | take 10`.

# Fabric KQL Table and JSON Mapping

---

## KQL - representative mapping from repository

```
.create table ShipmentEvents (  
    event_id:string,  
    order_id:long,  
    event_type:string,  
    event_ts:datetime,  
    region:string,  
    revenue:real,  
    fulfillment_minutes:long,  
    producer_seq:long  
)  
  
.create-or-alter table ShipmentEvents ingestion json mapping  
'ShipmentEventsJsonMapping'  
'[  
    {"column":"event_id","path":"$.event_id","datatype":"string"},  
    {"column":"order_id","path":"$.order_id","datatype":"long"},  
    {"column":"event_type","path":"$.event_type","datatype":"string"},  
    {"column":"event_ts","path":"$.event_ts","datatype":"datetime"},  
    {"column":"region","path":"$.region","datatype":"string"}  
']
```

# Fabric KQL Verification Queries

---

## KQL

```
ShipmentEvents  
| take 20
```

```
ShipmentEvents  
| summarize events=count() by bin(event_ts, 5m), event_type  
| order by event_ts desc
```

```
ShipmentEvents  
| where event_type == "DELAYED"  
| summarize delayed=count() by region  
| order by delayed desc
```

## Freshness and duplicate detection

## KQL

```
ShipmentEvents  
| summarize latest_event=max(event_ts)  
| extend freshness_seconds = datetime_diff("second", now(), latest_event)
```

# Fabric KQL Verification Queries - Continued

---

```
ShipmentEvents  
| summarize copies=count() by event_id  
| where copies > 1  
| order by copies desc
```

# Fabric OneLake / Lakehouse Path

---

- Create a Lakehouse in the same Fabric workspace.
- Where your tenant/authentication supports it, create a shortcut to the curated ADLS Gold path rather than copying the entire dataset.
- If a shortcut is not practical, load only the tiny Gold dataset for the learning POC and document that limitation.
- Use a Fabric Notebook, Dataflow Gen2, or Pipeline to create a small curated table.
- Compare the curated Fabric totals against the Databricks Gold result.

## **Shortcut concept**

A OneLake shortcut exposes supported external data without creating an unnecessary second physical copy.

# Synapse Serverless SQL - Important Path Requirement

---

## Architecture fork

The repository Synapse SQL is written for Gold Parquet stored in ADLS. The screenshot-validated successful run stored Gold in a Databricks managed Volume. To execute Synapse exactly as designed, also export/copy Gold to the ADLS realtime/gold/shipment\_summary path and grant Synapse read access.

- Create an Azure Synapse workspace.
- Use the built-in serverless SQL pool only.
- Do not create a dedicated SQL pool for this POC.
- Ensure the signed-in identity has read permission on the target ADLS path.



# Synapse SQL - Smoke Test over Gold Parquet

---

## Synapse serverless SQL

```
SELECT TOP 100 *  
FROM OPENROWSET(  
    BULK 'https://<storage-account>.dfs.core.windows.net/  
        realtime/gold/shipment_summary/*.parquet',  
    FORMAT = 'PARQUET'  
) AS rows;
```

## Pass condition

Rows are returned from the ADLS Gold Parquet folder using the built-in serverless SQL pool.

# Synapse SQL - Cost-Aware Projection

---

## Synapse serverless SQL

```
SELECT
    region,
    total_orders,
    delayed_shipments,
    revenue
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/
        realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
)
WITH (
    region VARCHAR(50),
    total_orders BIGINT,
    revenue FLOAT,
    delayed_shipments BIGINT
) AS gold
ORDER BY region;
```

# Synapse SQL - Cost-Aware Projection - Continued

---

## Cost lesson

Inspect data processed in query details and compare a broad `SELECT *` with the narrower projection. The POC dataset is tiny, so the measured difference may also be tiny.

# Synapse SQL - Business Totals

---

## Synapse serverless SQL

```
SELECT
    SUM(total_orders) AS total_orders,
    ROUND(SUM(revenue), 2) AS revenue,
    SUM(delayed_shipments) AS delayed_shipments
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/
        realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
)
WITH (
    total_orders BIGINT,
    revenue FLOAT,
    delayed_shipments BIGINT
) AS gold;
```

# Semantic Model Measures

---

## DAX

```
Total Orders =  
SUM('shipment_summary_fabric'[total_orders])
```

```
Revenue =  
SUM('shipment_summary_fabric'[revenue])
```

```
Delayed Shipments =  
SUM('shipment_summary_fabric'[delayed_shipments])
```

```
Average Fulfillment Time =  
AVERAGE('shipment_summary_fabric'[avg_fulfillment_minutes])
```

- Create four KPI cards/measures.
- Compare the totals with your authoritative curated Gold layer.
- If Eventhouse raw data contains duplicates, do not expect raw real-time totals to match deduplicated Gold automatically.

# Monitoring Checklist

---

| Area              | What to capture                                                        |
|-------------------|------------------------------------------------------------------------|
| Event Hubs        | Incoming Messages, Incoming Bytes, throttling/errors if any            |
| Databricks stream | Rows processed/written, batch duration, query status, checkpoint path  |
| Fabric            | Eventstream source/destination health and latest KQL event timestamp   |
| Freshness         | Maximum event_ts compared with current time                            |
| Transforms        | At least one successful run and one captured/recovered failure         |
| SQL               | Gold query result plus data-scanned/processed evidence where available |

# Failure and Recovery Evidence - What to Save

---

- Local producer failure showing the malformed/blank connection-string issue.
- Successful producer output.
- Successful receiver output.
- Databricks missing-secret error.
- Serverless CONFIG\_NOT\_AVAILABLE/JVM error with secret material redacted.
- Intermediate zero-row validation state.
- Successful 50-row Bronze write.
- Final 48-row Silver / 0-duplicate validation.
- Successful Gold export.
- SQL view/query evidence.

## Security

Never save or publish screenshots containing storage keys, SAS tokens, Event Hubs connection strings, or personal account identifiers. Rotate any credential that was exposed during troubleshooting.

# Purview / Governance Reference

---

- Purview is optional in this POC because account availability, permissions, and cost vary.
- If available, register only the relevant POC sources and scan the smallest practical scope.
- Review discovered assets, schema, classifications, and lineage where connectors expose it.
- If Purview is not deployed, document the intended enterprise design and clearly mark it design-only.
- Catalog/governance does not replace runtime access control: Azure RBAC/ACLs, Fabric permissions, Databricks Unity Catalog, and SQL permissions still matter.



# Common Troubleshooting Reference

---

| Symptom                                   | Check / fix                                                                           |
|-------------------------------------------|---------------------------------------------------------------------------------------|
| azure.eventhub module missing             | Activate .venv and pip install -r requirements.txt                                    |
| Receiver prints nothing                   | Send data, verify Event Hub name/connection, try starting-position -1                 |
| Kafka auth error                          | Namespace, :9093, SASL PLAIN, SASL_SSL, Listen rights, full connection string         |
| dropDuplicatesWithinWatermark unavailable | Use withWatermark(...).dropDuplicates(["event_id"]) and document runtime difference   |
| Cannot write abfss://                     | Check storage auth/RBAC/external location/credential and exact path                   |
| Synapse OPENROWSET denied                 | Check ADLS read permission, identity, dfs.core.windows.net path and file existence    |
| Fabric Eventstream empty                  | Verify Event Hubs messages, connection, source, destination mapping and Publish state |

# Cleanup - Stop Cost When the POC Is Finished

---

## Before deleting

- Save screenshots and query results.
- Record validation counts and Gold output.
- Stop Databricks compute / SQL warehouse.
- Save Fabric evidence if that branch was executed.

## Delete Azure POC resources

### Azure CLI option from the raw run notes

```
az group delete --name rg-logistics-poc --yes --no-wait
```

- Or delete the dedicated resource group from Azure portal.
- Delete Fabric POC items/workspace separately if they are not tied to the Azure resource group.
- Delete Purview/Log Analytics only if those resources were created solely for this POC and are not shared.

# Exact End-to-End Command Sequence - First Repeat Run

---

## Runbook

# 1. Local environment

```
python -m venv .venv  
.venv\Scripts\activate  
pip install -r requirements.txt  
copy .env.example .env
```

# 2. After .env is populated

```
python producer\send_events.py --count 20 --delay 0.2  
python producer\receive_events.py --max-events 10 --starting-position -1
```

# 3. Data-quality batch

```
python producer\send_events.py --count 200 --delay 0.15 \  
    --duplicate-every 25 --late-every 40
```

# 4. For Serverless availableNow, send before launching the sweep

```
python producer\send_events.py --count 50
```

# 5. In Databricks run in order

```
stream_to_delta.py  
inspect_and_validate.py
```

# Exact End-to-End Command Sequence - First Repeat Run - Continue

---

```
export_gold_parquet.py
```

```
# 6. Run Databricks SQL view/query statements
```

```
# 7. Execute Fabric/Synapse branches if required
```

# Final Validation Checklist

---

| Component      | Done when...                                                             |
|----------------|--------------------------------------------------------------------------|
| Resource group | POC resources are grouped together                                       |
| Event Hubs     | Producer sends and receiver reads                                        |
| Bronze         | Raw stream write succeeds and files/Delta log exist                      |
| Silver         | Rows > 0 and duplicate event_id count is 0                               |
| Checkpoint     | Restart uses the same working checkpoint and resumes correctly           |
| Gold           | Regional KPI Parquet is exported                                         |
| Databricks SQL | Silver/Gold view queries run successfully                                |
| Fabric         | If executed: Eventstream -> Eventhouse and KQL queries work              |
| Synapse        | If executed: serverless OPENROWSET reads ADLS Gold                       |
| Monitoring     | Arrival, stream health, freshness, and one failure/recovery are captured |
| Cleanup        | Compute stopped and POC-only resources removed when no longer needed     |

# Interview Q&A - Streaming Concepts

---

## **Q1. What is the Event Hubs partition-key trade-off?**

Using a stable partition key can preserve ordering for related events, but a highly skewed key can create a hot partition and uneven throughput.

## **Q2. What is a watermark?**

A watermark is an event-time threshold used by Structured Streaming to bound how long state is kept while waiting for late data.

## **Q3. Checkpoint vs watermark?**

Checkpoint persists stream progress/recovery state. Watermark controls event-time lateness/state retention. They solve different problems.

## **Q4. Why did Bronze have 50 rows while Silver had 48?**

The producer intentionally created duplicate event\_id values. Bronze retained the raw attempts; Silver applied deduplication and produced the clean curated count.

# Interview Q&A - Serving, Fabric and Governance

---

## **Q5. Eventhouse/KQL vs Lakehouse?**

Eventhouse/KQL is optimized for fast event/time-series ingestion and low-latency queries. A Lakehouse is oriented toward open lake storage, Spark/SQL analytics, and broader batch/ML/BI workflows.

## **Q6. When would you use Synapse serverless SQL?**

For on-demand SQL over lake files when you do not want to provision a dedicated SQL pool for the workload.

## **Q7. OneLake shortcut vs copy?**

A shortcut references accessible data without an unnecessary physical duplicate. A copy creates another dataset with its own storage and refresh lifecycle.

## **Q8. How do you monitor freshness?**

Track the latest business event timestamp and compare it with current time, then combine that signal with source-ingestion and stream-processing health.

# Beginner Completion Summary

---

- Start simple: prove Event Hubs locally before adding Databricks.
- Keep credentials out of code and screenshots; use secrets/managed identities for durable implementations.
- On Databricks Serverless, use governed storage patterns rather than runtime `fs.azure.account.key` configuration.
- Understand `availableNow=True`: it processes what is available and then finishes.
- Keep checkpoints for true restart tests; use a fresh checkpoint only for an intentional clean troubleshooting experiment.
- Treat Bronze as raw evidence, Silver as clean data, and Gold as business-ready aggregates.
- If you want Synapse serverless to query the result, ensure the Gold Parquet exists in ADLS rather than only in a Databricks managed Volume.
- Stop compute and clean up resources when the learning exercise is complete.

## Completed evidence result

The supplied POC evidence reached a successful 50-row Bronze write, 48 clean Silver events, duplicate count 0, Gold KPI export, and Databricks SQL view creation.